

MCU driver user guide

TRAVEO™ II family

About this document

Scope and purpose

This guide describes the architecture, configuration, and usage of the MCU driver. This guide also explains the functionality of the driver and provides a reference to the driver's API.

The installation, build process, and general information about the use of the EB tresos Studio are not within the scope of this document. See the *EB tresos Studio for ACG8 user's guide* [7] for detailed information on these topics.

Intended audience

This document is intended for anyone who uses the MCU driver of the TRAVEO™ II family.

Document structure

Chapter **1 General overview** gives a brief introduction to the MCU driver, explains the embedding of the driver in the AUTOSAR environment, and describes the supported hardware and development environment.

Chapter **2 Using the MCU driver** provides detailed steps required to use the MCU driver in the application.

Chapter **3 Structure and dependencies** describes the file structure and the dependencies for the MCU driver.

Chapter **4 EB tresos Studio configuration interface** describes the driver's configuration with the EB tresos Studio software.

Chapter **5 Functional description** gives a functional description of all services offered by the MCU driver.

Chapter **6 Hardware resources** describes the hardware resources used by the driver.

The **Appendix A** and **Appendix B** provides the complete API reference and access register table.

Abbreviations and definitions

Abbreviation	Definition
AHB	Advanced High-performance Bus
ALTHF	Alternate High-frequency clock
ALTTF	Alternate Low-frequency clock
ASIL	Automotive Safety Integrity Level
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basic Software. Standardized part of software which does not fulfill a vehicle functional job.
BOD	Brown-out Detection
CCO	Current Controlled Oscillator
CM0+	Cortex®-M0+ processor

About this document

Abbreviation	Definition
CM4	Cortex®-M4 processor
CM7	Cortex®-M7 processor
DEM	Diagnostic Event Manager
DET	Default Error Tracer
DMA	Direct Memory Access
DSI	Digital System Interconnect
EB tresos ECU AUTOSAR Suite	A collection of AUTOSAR Basic Software modules and a Runtime Environment integrated in a common configuration and build environment.
EB tresos Studio	Elektrobit Automotive configuration framework
ECO	External Crystal Oscillator
FLL	Frequency Locked Loop
HF clock	High-frequency clock
HVLVD	High Voltage / Low Voltage Detector
ILO	Internal Low-speed Oscillator
ISR	Interrupt Service Routine
LF clock	Low-frequency clock
LPECO	Low-power External Crystal Oscillator
LVD	Low Voltage Detector
IMO	Internal Main Oscillator
MCAL	Microcontroller Abstraction Layer
MCU	Microcontroller Unit
OCD	Over-current Detection
OS	Operating System
OVD	Over-voltage Detection
PCLK	Programmable Clock
PFD	Phase Frequency Detector
PLL	Phase Locked Loop
PMIC	Power Management Integrated Circuit
RAM	Random Access Memory
REGHC	High-current Regulator
ROM	Read-Only Memory
RTC	Real-Time Clock
SSCG	Spread Spectrum Clock Generator
VADJ	Voltage Adjustment
WCO	Watch Crystal Oscillator
WDT	Watchdog Timer
WFI	Arm® Wait For Interrupt instruction
μC	Microcontroller

Related documents

AUTOSAR requirements and specifications

Bibliography

- [1] General specification of basic software modules, AUTOSAR release 4.2.2
- [2] AUTOSAR specification of MCU driver, release 4.2.2.
- [3] AUTOSAR specification of standard types, release 4.2.2.
- [4] Specification of ECU configuration parameters, AUTOSAR release 4.2.2.
- [5] AUTOSAR specification of default error tracer, release 4.2.2.
- [6] AUTOSAR specification of diagnostics event manager, release 4.2.2.

Elektrobit automotive documentation

Bibliography

- [7] EB tresos Studio for ACG8 user's guide.

Hardware documentation

The hardware documents are listed in the delivery notes.

Related standards and norms

Bibliography

- [8] AUTOSAR layered software architecture, release 4.2 revision 2.

Table of contents

About this document.....	1
Table of contents.....	4
1 General overview	7
1.1 Introduction to the MCU driver	7
1.2 User profile	7
1.3 Embedding in the AUTOSAR environment.....	7
1.4 Supported hardware	8
1.5 Development environment.....	8
1.6 Character set and encoding	8
2 Using the MCU driver	9
2.1 Installation and prerequisites.....	9
2.2 Configuring the MCU driver.....	9
2.2.1 Architecture specifics.....	10
2.3 Adapting an application.....	10
2.4 Starting the build process.....	11
2.5 Measuring stack consumption.....	12
2.6 Memory mapping	12
2.6.1 Memory allocation keyword	12
3 Structure and dependencies	14
3.1 Static files	14
3.2 Configuration files	14
3.3 Generated files	14
3.4 Dependencies	15
3.4.1 DET.....	15
3.4.2 DEM.....	15
3.4.3 AUTOSAR OS.....	15
3.4.4 BSW scheduler.....	15
3.4.5 Error callout handler	15
4 EB tresos Studio configuration interface.....	16
4.1 General configuration	16
4.2 MCU module configuration.....	16
4.3 MCU low voltage detection callback functions.....	18
4.4 MCU DEM event parameter references.....	18
4.5 MCU clock setting configuration.....	18
4.5.1 MCU clock input	19
4.5.1.1 MCU ECO clock settings	20
4.5.1.2 MCU ECO prescaler settings	20
4.5.1.3 MCU ECO trim settings.....	20
4.5.1.4 MCU LPECO clock settings.....	21
4.5.1.5 MCU LPECO prescaler settings	22
4.5.1.6 MCU ILO clock settings.....	22
4.5.1.7 MCU ILO1 clock settings.....	22
4.5.1.8 MCU WCO clock settings	23
4.5.2 MCU clock settings	23
4.5.2.1 MCU clock path settings	26
4.5.2.2 MCU FLL clock settings	26
4.5.2.3 MCU PLL clock settings	28

Table of contents

4.5.2.4	MCU SSCG PLL clock settings	29
4.5.2.5	MCU clock root settings	32
4.5.2.6	MCU PCLK group settings	33
4.5.2.7	MCU PCLK divider settings.....	33
4.5.2.8	MCU PCLK settings	34
4.5.2.9	MCU peripheral group settings.....	34
4.5.2.10	MCU peripheral group slave settings	35
4.5.2.11	MCU LF clock settings	35
4.5.2.12	MCU pump clock settings	36
4.5.2.13	MCU timer clock settings	37
4.5.2.14	MCU clock output settings.....	37
4.5.2.15	MCU clock supervisor settings.....	39
4.5.3	MCU clock reference point.....	40
4.6	MCU mode settings configuration	40
4.6.1	MCU hibernate mode settings	44
4.6.2	MCU HVLVD settings.....	44
4.6.3	MCU supply supervision settings.....	45
4.6.4	MCU REGHC settings	46
4.6.5	MCU PMIC settings	46
4.6.6	MCU DMA settings	47
4.7	MCU RAM section configuration	47
4.8	MCU published information.....	48
5	Functional description	49
5.1	Inclusion	49
5.2	Initialization.....	49
5.3	MCU mode	50
5.4	API parameter checking.....	50
5.5	Production error detection	51
5.6	Reentrancy.....	51
5.7	Debugging support.....	51
5.8	APIs requiring privileged execution.....	51
6	Hardware resources	52
6.1	Timer	52
6.2	Interrupts.....	52
6.3	Fault report structure.....	53
7	Appendix A – API reference	54
7.1	Data types.....	54
7.1.1	Mcu_ConfigType.....	54
7.1.2	Mcu_PllStatusType	54
7.1.3	Mcu_ClockType	54
7.1.4	Mcu_ResetType	54
7.1.5	Mcu_RawResetType	55
7.1.6	Mcu_ModeType	55
7.1.7	Mcu_RamSectionType	55
7.1.8	Mcu_RamStateType	55
7.1.9	Mcu_StatusType.....	55
7.1.10	Mcu_CpuStatusType	56
7.1.11	Mcu_SysStatusType	56
7.2	Constants.....	56
7.2.1	Error codes	56

Table of contents

7.2.2	Version information	57
7.2.3	Module information	57
7.2.4	API service IDs	57
7.3	Functions	58
7.3.1	Mcu_Init	58
7.3.2	Mcu_InitRamSection	58
7.3.3	Mcu_InitClock	59
7.3.4	Mcu_DistributePllClock	59
7.3.5	Mcu_GetPllStatus	60
7.3.6	Mcu_GetResetReason	61
7.3.7	Mcu_GetResetRawValue	61
7.3.8	Mcu_PerformReset	62
7.3.9	Mcu_SetMode	62
7.3.10	Mcu_GetVersionInfo	63
7.3.11	Mcu_CheckClockStatus	64
7.3.12	Mcu_CheckModeStatus	64
7.4	Required callback functions	65
7.4.1	DET	65
7.4.1.1	Det_ReportError	65
7.4.2	DEM	66
7.4.2.1	Dem_ReportErrorStatus	66
7.4.3	Callout functions	66
7.4.3.1	Error callout API	66
8	Appendix B – Access register table	68
8.1	PERI	68
8.2	CPUSS	70
8.3	DW	75
8.4	DMAC	75
8.5	FLASHC	76
8.6	SRSS	76
8.7	BACKUP	87
8.8	CM0P_SCS	89
8.9	CM4_SCS	90
8.10	CM7_SCS	90
	Revision history	92

1 General overview

1.1 Introduction to the MCU driver

The MCU driver is a set of software routines for initializing the MCU, and provides configuration options for the following:

- Clock settings
- Low-Power modes
- RAM section initialization

The driver is compliant with the AUTOSAR standard and is implemented according to *AUTOSAR specification of MCU driver* [2].

In addition, the MCU driver is delivered with a plugin for the EB tresos Studio software, which allows you to statically configure the driver options. The driver also provides an interface to define symbolic names and the functionality of all configuration options.

1.2 User profile

This guide is intended for users with a basic knowledge of the following domains:

- Embedded systems
- C programming language
- The AUTOSAR standard
- The target hardware architecture

1.3 Embedding in the AUTOSAR environment

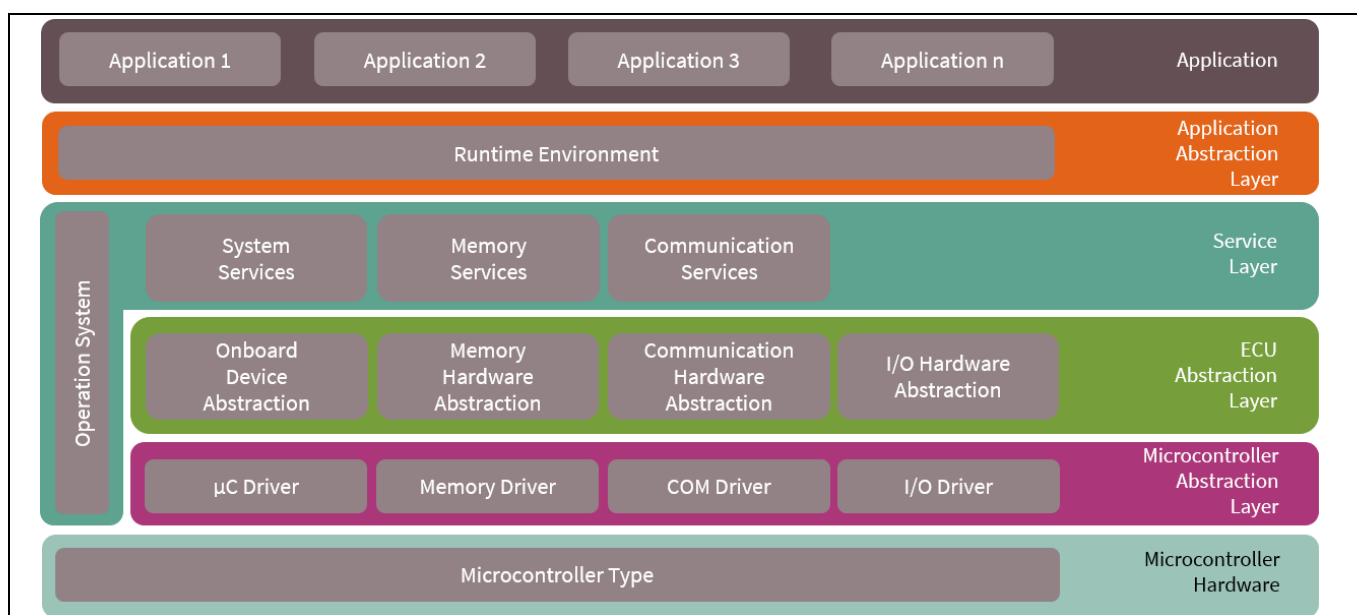


Figure 1 Overview of AUTOSAR software layers

Figure 1 shows the layered AUTOSAR software architecture. The MCU driver (**Figure 2**) is part of the microcontroller abstraction layer (MCAL), the lowest layer of basic software in the AUTOSAR environment.

As an internal I/O driver, the MCU driver provides a standardized and μ C-independent interface to higher software layers for accessing clocks and CPU modes of the ECU hardware.

For an overview of the AUTOSAR layered software architecture, see *AUTOSAR layered software architecture* [8].

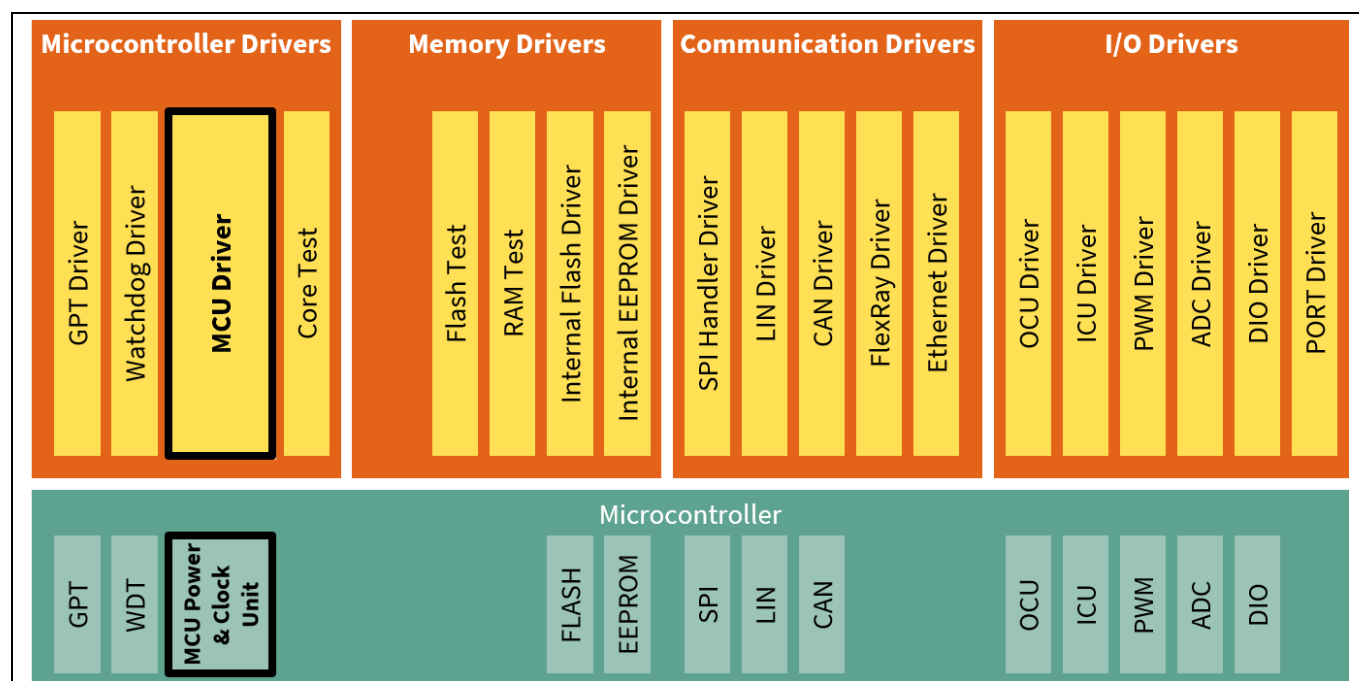


Figure 2 MCU driver in MCAL layer

1.4 Supported hardware

This version of the MCU driver supports the TRAVEO™ II microcontroller family. The supported derivatives are listed in the release notes.

Additional derivatives that contain only a subset of the capabilities of one derivative mentioned above can be implemented and supported by providing a resource file with its properties.

1.5 Development environment

The development environment corresponds to AUTOSAR release 4.2.2. The Base, Make, and Resource modules are required for the proper functionality of the MCU driver.

1.6 Character set and encoding

All source code files of the MCU driver are restricted to the ASCII character set. The files are encoded in UTF-8 format, with only the 7-bit subset (values 0x00 ... 0x7F) is used.

2 Using the MCU driver

This chapter describes all necessary steps to incorporate the MCU driver into your application.

2.1 Installation and prerequisites

Note: Before you start, see the *EB tresos Studio for ACG8 user's guide* [7] for the following information.

- The installation procedure of EB tresos ECU AUTOSAR components.
- The usage of the EB tresos Studio.
- The usage of the EB tresos ECU AUTOSAR build environment (It includes an explanation of how to setup and integrate the own application within the EB tresos ECU AUTOSAR build environment).

The installation of the MCU driver complies with the general installation procedure for EB tresos ECU AUTOSAR components given in the documents mentioned above. If the driver has been successfully installed, the driver will appear in the module list of the EB tresos Studio (see *EB tresos Studio for ACG8 user's guide* [7]).

This document assumes that the project is properly set up and is using the application template as described in the *EB tresos Studio for ACG8 user's guide* [7]. This template provides the necessary folder structure, project, and makefiles needed to configure and compile an application within the build environment. You need to be familiar with the usage of the command line shell.

2.2 Configuring the MCU driver

This section provides a brief overview of the configuration structure defined by AUTOSAR to use the MCU driver.

The following three basic containers are used to configure the common behavior.

- `McuGeneralConfiguration`: This container is mainly used to restrict/extend the API of the MCU module and enable/disable default error trace.
- `McuModuleConfiguration`: This container is mainly used for clock settings, the RAM initialization settings, and the low-power mode settings of the MCU module.
- `McuPublishedInformation`: This container holds the value of the cause of reset supported in the MCU.

For detailed information and description, see [EB tresos Studio configuration interface](#).

Note: Ensure that the application also includes an AUTOSAR-compliant DET when default error detection is enabled; otherwise the application will not compile.

You must set up the following characteristics for each MCU configuration:

- Clock configurations
- Number of RAM sectors
- RAM sector configurations
- Number of low-power modes
- Low-power mode configurations

The `McuGeneralConfiguration` container describes the individual MCU setup information.

2.2.1 Architecture specifics

- `McuSafetyFunctionApi`: Adds or removes the `Mcu_CheckClockStatus()` and `Mcu_CheckModeStatus()` services from the code.
- `McuErrorCalloutFunction`: Specifies the error callout handler that is called when errors are detected during runtime.
- `McuIncludeFile`: Specifies the file name to include definitions such as declaration for error callout handler.
- `McuModuleConfiguration`: Contains architecture-specific parameters. See [MCU module configuration](#).

2.3 Adapting an application

To use the MCU driver in an application, do the following:

Step 1: Include the MCU driver header file by adding the following line of code to the source file:

```
#include "Mcu.h" /* MCU Driver */
```

This publishes all needed function and data prototypes and symbolic names of the configuration to the application.

Step 2: Implement the error callout function for ASIL safety extension.

To do this, declare the error callout function in the file specified by `McuIncludeFile` and implement it in your application (see [Required callback functions](#), Error callout API).

The error callout function name can be configured by the `McuErrorCalloutFunction` parameter.

Step 3: Initialize and configure the MCU.

See [EB tresos Studio configuration interface](#). The MCU module will automatically be enabled if an appropriate parameter configuration of the MCU module is available in the application.

The MCU initialization can be done with the following function call and parameter.

```
Mcu_Init(&Mcu_Config[0]);
```

As part of the initialization process, call the `Mcu_InitClock` API.

The following is a short sample for a clock `MY_CLOCK` configured as a clock setting and for a mode `MY_MODE` configured as a mode setting:

```
Mcu_InitClock(McuConf_McuClockSettingConfig_MY_CLOCK);
```

An additional call of the next API function might be needed (depending on the underlying hardware) to set up the clock properly:

```
Mcu_DistributePllClock();
```

If RAM sectors are configured, they need to be initialized with a separate call of the API function, stated below, for each RAM sector configuration set:

```
Mcu_InitRamSection(RamSectorConfigurationID);
```

All other APIs (except for `Mcu_CheckClockStatus`, `Mcu_CheckModeStatus`) calls might be used after successful initialization of the MCU whenever necessary. These functions are:

```
Mcu_GetPllStatus();
```

```
Mcu_GetResetRawValue();
```

```
Mcu_GetResetReason();  
Mcu_PerformReset();  
Mcu_GetVersionInfo(&versioninfo);  
Mcu_SetMode(McuConf_McuModeSettingConf_MY_MODE);
```

Note: *Since power mode is needed to be controlled on each CPU core when entering system Sleep or DeepSleep mode, it is necessary to make it possible for the MCU driver to run on each CPU core.*

Your application must provide the notification functions and its declarations that you configured. The file containing the declarations must be included using `McuGeneralConfiguration/McuIncludeFile`. The notification functions take no parameters and have void return type:

```
void MyNotificationFunction(void)  
{  
/* Insert your code here */  
}
```

The notification function is called from an interrupt context.

2.4 Starting the build process

Do the following to build your application:

Note: *For a clean build, use the build command with target `clean_all` before (`make clean_all`).*

1. On the command shell, type the following command to generate the necessary configuration-dependent files. See [Generated files](#).

```
> make generate
```

2. Type the following command to resolve the required file dependencies

```
> make depend
```

3. Type the following command to compile and link the application

```
> make (optional target: all)
```

The application is now built. All files are compiled and linked to a binary file, which can be downloaded to the target hardware.

Note: *MCU driver must be located on all CPU cores to enter low-power mode. In this case, MCU driver should be built for all CPU cores.*

2.5 Measuring stack consumption

Do the following to measure stack consumption. It requires the Base module for proper measurement.

Note: All files (including library files) should be rebuilt with the dedicated compiler option. The executable file built in this step must be used only to measure stack consumption.

1. Add the following compiler option to the Makefile to enable stack consumption measurement.

```
-DSTACK_ANALYSIS_ENABLE
```

2. Type the following command to clean library files.

```
> make clean_lib
```

3. Follow the build process described in [Starting the build process](#).
4. Follow the instructions in the release notes and measure the stack consumption

2.6 Memory mapping

The *Mcu_MemMap.h* file in the $\$(TRESOS_BASE)/plugins/MemMap_TS_T40D13M0I0R0$ directory is a sample. This sample file is replaced by the file generated by the MEMMAP module. Input to the MEMMAP module is generated as *Mcu_Bswmd.arxml* in the $\$(PROJECT_ROOT)/output/generated/swcd$ directory of your project folder.

2.6.1 Memory allocation keyword

- `MCU_START_SEC_CODE_ASIL_B / MCU_STOP_SEC_CODE_ASIL_B`

The memory section type is CODE. All executable code is allocated in this section.

- `MCU_START_SEC_CONST_ASIL_B_UNSPECIFIED / MCU_STOP_SEC_CONST_ASIL_B_UNSPECIFIED`

The memory section type is CONST. The following constants are allocated in this section:

- All configuration data except reset.
- Hardware register base address data.
- `MCU_START_SEC_CONST_ASIL_B_32 / MCU_STOP_SEC_CONST_ASIL_B_32`

The memory section type is CONST. The following constants are allocated in this section:

- Reset configuration data.
- `MCU_START_SEC_VAR_INIT_ASIL_B_UNSPECIFIED / MCU_STOP_SEC_VAR_INIT_ASIL_B_UNSPECIFIED`

The memory section type is VAR. The following variables are allocated in this section:

- Pointer to the configuration data.
- Pointer to the hardware register base address data.
- `MCU_START_SEC_VAR_INIT_ASIL_B_BOOLEAN / MCU_STOP_SEC_VAR_INIT_ASIL_B_BOOLEAN`

The memory section type is VAR. The following variable is allocated in this section:

- Clock setting status.

- `MCU_START_SEC_VAR_INIT_ASIL_B_8 / MCU_STOP_SEC_VAR_INIT_ASIL_B_8`

The memory section type is VAR. The following variables are allocated in this section:

- Driver status.
- Clock ID set to the hardware.
- `MCU_START_SEC_VAR_NO_INIT_ASIL_B_UNSPECIFIED /`
`MCU_STOP_SEC_VAR_NO_INIT_ASIL_B_UNSPECIFIED`

The memory section type is VAR. The following variables are allocated in this section:

- Reset reason.
- Reset raw value.

3 Structure and dependencies

The MCU driver consists of static, configuration, and generated files.

3.1 Static files

- $\$(PLUGIN_PATH)=\$(TRESOS_BASE)/plugins/MCU_TS_*$ is the path to the MCU module plugin.
- $\$(PLUGIN_PATH)/lib_src$ contains all static source files of the MCU driver. These files represent the functionality of the driver; therefore, the files are independent of any configuration sets.
- $\$(PLUGIN_PATH)/src$ contains configuration-dependent source files or special derivative files. Each file is rebuilt when the configuration set is changed.

All necessary source files will be automatically compiled and linked during the build process and all include paths will be set if the MCU driver is enabled.

- $\$(PLUGIN_PATH)/include$ is the basic public include directory that you need to include in *Mcu.h*.
- $\$(PLUGIN_PATH)/autosar$ directory contains the AUTOSAR ECU parameter definition with vendor, architecture, and derivative-specific adaptations to create a correct matching parameter configuration for the MCU module.

3.2 Configuration files

The configuration of the MCU driver is done with the EB tresos Studio software. When saving a project, the configuration description is written to the *Mcu.xdm* file. It is located under $\$(PROJECT_ROOT)/config$ in your project folder. This file serves as the input to generate the configuration-dependent source and header files during the build process.

3.3 Generated files

During the build process, the following files are generated based on the current configuration description, and are in the *output/generated* subfolder of your *project* folder:

- *include/Mcu_Cfg.h* provides settings of configurations with pre-compile attribute; for example, all symbolic names required by the API for clock, RAM sector, and low-power mode configurations. In addition, this file defines a `DemEventId` parameter of the DEM module, which is referred in the configuration. The DEM module is included by *Mcu.h*.
- *include/Mcu_Cfg_Arch.h* provides architecture-specific settings of configurations with pre-compile attribute; for example, each hardware IP register base address.
- *include/Mcu_PBCfg.h* provides settings of configurations with post-build attribute; for example, symbolic names of module configurations. In addition, it defines the number of module, clock, RAM sector, and low-power mode configurations.
- *include/Mcu_PBCfg_Arch.h* provides architecture-specific settings of configurations with post-build attribute.
- *include/Mcu_ExternalInclude.h* includes the header files specified by *McuIncludeFile*.
- *src/Mcu_PBCfg.c* contains the constants for the MCU configuration.
- *src/Mcu_Irq.c* contains the interrupt service routine.

Note: *Generated source files need not to be added to your application make file. They are compiled and linked automatically during the build process. Check the consistency of the configuration and generated files.*

swcd/Mcu_Bswmd.arxml contains BSW module description.

Note: Additional steps are required for the generation of BSW module description.

In EB tresos Studio, follow the menu path Project > Build Project and select generate_swcd.

3.4 Dependencies

3.4.1 DET

If the default error detection is enabled in the MCU driver module configuration, DET must be installed, configured, and integrated into the application.

3.4.2 DEM

If clock failure notification or reset failure notification is enabled in the MCU driver module configuration, DEM must be installed, configured, and integrated into the application.

3.4.3 AUTOSAR OS

The OS must be used to configure and to create the ISR vector table entries for the MCU driver. See [Interrupts](#) for more information.

3.4.4 BSW scheduler

The MCU driver uses the following services of the BSW scheduler (originally named SchM, now BswM) to enter and leave critical sections:

- SchM_Enter_Mcu_MCU_EXCLUSIVE_AREA_0(void)
- SchM_Exit_Mcu_MCU_EXCLUSIVE_AREA_0(void)

Make sure that the BSW scheduler is properly configured and initialized before using the MCU driver services.

3.4.5 Error callout handler

The error callout handler is called on every error that is detected regardless of whether default error detection is enabled or disabled. The error callout handler is an ASIL safety extension that is not specified by AUTOSAR. It is configured via configuration parameter `McuErrorCalloutFunction`.

4 EB tresos Studio configuration interface

The GUI is not part of this delivery. For further information, see *EB tresos Studio for ACG8 user's guide* [7].

4.1 General configuration

The `McuGeneralConfiguration` container has the following parameters to configure the general functions of the MCU driver:

- `McuDevErrorDetect` enables or disables the development error notification feature for the MCU driver module.

Setting this parameter to `FALSE` disables the notification of development errors via DET. However, in contrast to the AUTOSAR specification, detection of development errors is still enabled as a safety mechanism (fault detection).

- `McuGetRamStateApi` is not used and is not being evaluated.
- `McuInitClock` enables or disables the clock initialization functionality.
- `McuNoPll` enables or disables the functionality of the PLL clock.
- `McuPerformResetApi` enables or disables the reset functionality.
- `McuVersionInfoApi` enables or disables the functionality to read the module version information.
- `McuSafetyFunctionApi` adds or removes the `Mcu_CheckClockStatus()` and `Mcu_CheckModeStatus()` services from the code.
- `McuErrorCalloutFunction` is used to specify the error callout function name. The function is called on every error. The ASIL level of this function limits the ASIL level of the MCU driver.

Note: *`McuErrorCalloutFunction` must be a valid C function name; otherwise an error would occur in the configuration phase.*

- `McuIncludeFile` lists the file names that will be included within the driver. Any application-specific symbol that is used by the MCU driver module configuration (such as error callout function) should be included by configuring this parameter.

Note: *`McuIncludeFile` must be a filename with the `.h` extension and a unique name; otherwise some errors would occur in the configuration phase.*

4.2 MCU module configuration

The `McuModuleConfiguration` container has the following the parameters to configure the microcontroller specific functions:

- `McuClockSrcFailureNotification` enables or disables the clock failure notification to DEM.
 - `DISABLED`: Disables the clock failure notification to DEM.
 - `ENABLED`: Enables the clock failure notification to DEM.
- `McuResetFailureNotification` enables or disables the reset failure notification to DEM.
- `McuNumberOfMcuModes` specifies the number of modes configured.
- `McuRamSectors` specifies the number of RAM sectors configured.
- `McuResetSetting` is not used; instead, the architecture-specific parameter `McuResetSelect` is used.
- `McuResetSelect` specifies the reset type:

- `MCU_SW_RESET`: Software reset: This parameter relates to the MCU-specific reset configuration. It applies to the `Mcu_PerformReset()` function, which performs a microcontroller reset using the hardware function of the microcontroller.
- `McuEnableCacheFlushBeforeReset` enables or disables flushing cache before performing reset.

Note: *`McuEnableCacheFlushBeforeReset` is available only if `McuPerformResetApi` is `TRUE`, `McuResetSelect` is activated.*

- `McuRamWriteBufferTimeoutBeforeReset` specifies the timeout count value used when checking whether the RAM write buffer is empty.
 - 1 - 4294967295: Timeout count value used when checking that the RAM write buffer is empty.
- `McuForcedResetEnable` enables or disables performing reset even if an error occurs in `Mcu_PerformReset()` function.

Note: *`McuForcedResetEnable` is available only if `McuPerformResetApi` is `TRUE` and `McuResetSelect` is activated.*

- `McuRam0Macro<n>RetainBeforeReset` (<n> = 0 ... 15) specifies whether to retain RAM0 Macro <n> during reset.

Note: *`McuRam0Macro<n>RetainBeforeReset` is available only if `McuPerformResetApi` is `TRUE`, `McuResetSelect` is activated and RAM0 Macro <n> is supported by the derivative.*

- `McuRam1RetainBeforeReset` specifies whether to retain RAM1 during reset.

Note: *`McuRam1RetainBeforeReset` is available only if `McuPerformResetApi` is `TRUE`, `McuResetSelect` is activated and RAM1 is supported by the derivative.*

- `McuRam2RetainBeforeReset` specifies whether to retain RAM2 during reset.

Note: *`McuRam2RetainBeforeReset` is available only if `McuPerformResetApi` is `TRUE`, `McuResetSelect` is activated and RAM2 is supported by the derivative.*

- `McuClearResetReasonRegister` enables or disables clearing the reset reason registers during `Mcu_Init()`.

Note: *If `McuClearResetReasonRegister` is `FALSE`, you should initialize the following reset reason registers; otherwise the `Mcu_GetResetReason()` API would not be able to read the reset reason correctly.*

- `RES_CAUSE`
- `RES_CAUSE2`
- `McuEnableDefaultClock` initializes the clock during `Mcu_Init()` when checked. This has the advantage that subsequent calls to `Mcu_InitClock()` can be omitted and all subsequent initialization or startup operations benefit from the higher speed of the clock.

Note: *If `McuInitClock` is `FALSE`, this parameter should also be disabled.*

- `McuDefaultClockSetting` selects the default clock setting configuration from `McuClockSettingConfig`, which is used to initialize the clock automatically when the MCU is initialized (`Mcu_Init()`).

Note: *This parameter is available only if `McuEnableDefaultClock` is `TRUE`.*

MCU module configuration contains the following containers:

- `McuLowVoltageDetectionCallbackFunctions` (see [MCU low voltage detection callback functions](#))
- `McuDemEventParameterRefs` (see [MCU DEM event parameter references](#))
- `McuClockSettingConfig` (see [MCU clock setting configuration](#))
- `McuModeSettingConf` (see [MCU mode settings configuration](#))
- `McuRamSectorSettingConf` (see [MCU RAM section configuration](#))

4.3 MCU low voltage detection callback functions

The `McuLowVoltageDetectionCallbackFunctions` container has the following parameters to configure the callback functions for notifying an error from the low voltage detection:

- `McuHvLvd1Notification` specifies the name of the function for notifying the error from the HVLVD1.
- `McuHvLvd2Notification` specifies the name of the function for notifying the error from the HVLVD2.

Note: *Notifications must be declared and defined outside the MCU module. The file containing the declarations must be included using `McuIncludeFile`.*

4.4 MCU DEM event parameter references

The `McuDemEventParameterRefs` container has the following parameters to configure the DEM event notification settings:

- `MCU_E_CLOCK_FAILURE` refers to the configured DEM event to report "Clock source failure".
- `MCU_E_RESET_FAILURE` refers to the configured DEM event to report "Reset failure".

4.5 MCU clock setting configuration

The `McuClockSettingConfig` container has the following parameters to configure the clock settings:

- `McuClockSettingId` is a logical ID of the clock setting. This value will be assigned to the following symbolic names:
 - The symbolic name derived from the `McuClockSettingConfig` container short name is prefixed with "`McuConf_McuClockSettingConfig_`".

Example:

`McuConf_McuClockSettingConfig_McuClockSettingConfig_0.`

Note: *In the same `McuModuleConfiguration` container, `McuClockSettingId` must be unique and consecutive. `McuUnlockWatchdogEnable` enables or disables to unlock watchdog once before setting the LF clock and ILO0 clock.*

Note: *If `McuUnlockWatchdogEnable` is `FALSE`, setting of LF clock and ILO0 clock will be skipped when watchdog is locked.*

The MCU clock setting configuration holds the following containers.

- `McuClocksIn` (see [MCU clock input](#))
- `McuClockSettings` (see [MCU clock settings](#))
- `McuClockReferencePoint` (see [MCU clock reference point](#))

Note: The acceptable frequency range of each clock shown in the following sections depends on the subderivative. For more information about the derivative-dependent clock frequency, see the hardware technical reference manual or data sheet.

If each clock is disabled, its frequency will be set to 0.0 (in Hz).

As the number of clock configuration increases, the duration of critical section in the `Mcu_Init()`, `Mcu_InitClock()`, and `Mcu_SetMode()` will be longer.

4.5.1 MCU clock input

The `McuClocksIn` container has the following parameters to configure the input clocks:

- `McuImoEnable` enables or disables the IMO clock.

Note: This parameter must be set to `TRUE` at all times for all functions to work properly.

- `McuImoFrequency` specifies the frequency of the IMO clock (in Hz).

Note: If `McuImoEnable` is `FALSE`, this parameter must be set to 0.0 (in Hz).

- `McuExtFrequency` specifies the frequency of the external clock (in Hz).
- `McuAlthfFrequency` specifies the frequency of the ALTHF clock (in Hz).

Note: If ALTHF clock is not supported by the derivative, this parameter must be set to 0.0 (in Hz).

- `McuAltlfFrequency` specifies the frequency of the ALTLF clock (in Hz).

Note: If ALTLF clock is not supported by the derivative, this parameter must be set to 0.0 (in Hz).

- `McuDsiOut<n>Frequency` (<n> = 0 ... 15) specifies the frequency of the DSI output <n> clock (in Hz).

Note: `McuDsiOut<n>Frequency` is available only if each DSI mux is supported by the derivative.

MCU clock input configuration holds the following containers:

- `McuEcoSettings` (see [MCU ECO clock settings](#))
- `McuLpEcoSettings` (see [MCU LPECO clock settings](#))
- `McuIloSettings` (see [MCU ILO clock settings](#))
- `McuWcoSettings` (see [MCU WCO clock settings](#))

4.5.1.1 MCU ECO clock settings

The `McuEcoSettings` container has the following parameters to configure the ECO clock:

- `McuEcoEnable` enables or disables the ECO clock.
- `McuAgcEnable` enables or disables the automatic gain control.
- `McuEcoAmpStabilizationTimeout` specifies the timeout count value used when checking that the ECO clock is stabilized.
 - 1 - 4294967295: Timeout count value used when checking that the ECO clock is stabilized.

Note: *Even if `McuEcoAmpStabilizationTimeout` is deactivated, the ECO clock status will be checked once.*

- `McuEcoFrequency` specifies the frequency of the ECO clock oscillator (in Hz).

Note: *If `McuEcoEnable` is FALSE, this parameter must be set to 0.0 (in Hz).*

The MCU ECO clock settings configuration holds the following container:

- `McuEcoPrescalerSettings` (see [MCU ECO prescaler settings](#))
- `McuEcoTrimSettings` (see [MCU ECO trim settings](#))

4.5.1.2 MCU ECO prescaler settings

Use the following parameters to configure the ECO prescaler:

- `McuEcoPrescalerEnable` enables or disables the ECO prescaler.
- `McuEcoPrescalerValue` specifies the ECO prescaler value.
 - 1 - 1024.99609375: ECO prescaler value.
- `McuEcoPrescalerEnableTimeout` specifies the timeout count value used when checking that the ECO prescaler is enabled.
 - 1 - 4294967295: Timeout count value used when checking that the ECO prescaler is enabled.

Note: *Even if `McuEcoPrescalerEnableTimeout` is deactivated, the ECO prescaler status will be checked once.*

- `McuEcoPrescaledFrequency` specifies the frequency of the prescaled ECO clock (in Hz).

Note: *`McuEcoPrescaledFrequency` automatically displays the resulting frequency calculated by following formula: $McuEcoPrescaledFrequency = McuEcoFrequency / McuEcoPrescalerValue$*

4.5.1.3 MCU ECO trim settings

The `McuEcoTrimSettings` container has the following parameters to configure the ECO clock trim setting:

- `McuEcoAmplitudeTrimValue` specifies the ECO amplitude trim value to set the crystal drive level.
 - `MCU_ECO_AMPLITUDE_TRIM_VP_LESS_THAN_0_35V`: ECO amplitude trim when $V_p < 0.35$ [V].
 - `MCU_ECO_AMPLITUDE_TRIM_VP_LESS_THAN_0_40V`: ECO amplitude trim when $V_p < 0.40$ [V].
 - ...
- `McuEcoFeedbackResistorTrimValue` specifies the ECO feedback resistor trim value.
 - 0 - 3: ECO feedback resistor trim value.

- `McuEcoFilterTrimValue` specifies the ECO low-pass filter frequency trim value.
 - 0 - 3: ECO low-pass filter frequency trim value.
- `McuEcoGainTrimValue` specifies the ECO amplifier gain trim value.
 - 0 - 7: ECO amplifier gain trim value.
- `McuEcoWatchdogTrimValue` specifies the ECO watchdog trim value.
 - `MCU_ECO_WATCHDOG_TRIM_VP_GREATER_THAN_0_05V`: ECO watchdog trim when $V_p > 0.05$ [V].
 - `MCU_ECO_WATCHDOG_TRIM_VP_GREATER_THAN_0_10V`: ECO watchdog trim when $V_p > 0.10$ [V].
 - ...

4.5.1.4 MCU LPECO clock settings

The `McuLpEcoSettings` container has the following parameters to configure the LPECO clock:

- `McuLpEcoEnable` enables or disables the LPECO clock.
- `McuLpEcoStopForUpdate` enables or disables to stop the LPECO clock once before setting.

Note: *If `McuLpEcoStopForUpdate` is FALSE, setting of the LPECO clock will be skipped when it is running.*

- `McuLpEcoAmplitudeDetectorEnable` enables or disables the minimum amplitude detector for the LPECO clock.
- `McuLpEcoMaximumAmplitude` specifies the LPECO maximum oscillation amplitude.
 - `MCU_LPECO_AMPLITUDE_1_35V`: LPECO maximum oscillation amplitude 1.35[V].
 - `MCU_LPECO_AMPLITUDE_1_80V`: LPECO maximum oscillation amplitude 1.80[V].
- `McuLpEcoLoadCapacitanceRange` specifies the LPECO load capacitance range of the crystal.
 - `MCU_LPECO_LOAD_CAPACITANCE_TO_10PF`: LPECO load capacitance range [5pF, 10pF].
 - `MCU_LPECO_LOAD_CAPACITANCE_TO_15PF`: LPECO load capacitance range (10pF, 15pF].
 - `MCU_LPECO_LOAD_CAPACITANCE_TO_20PF`: LPECO load capacitance range (15pF, 20pF].
 - `MCU_LPECO_LOAD_CAPACITANCE_TO_25PF`: LPECO load capacitance range (20pF, 25pF].
- `McuLpEcoAmpStabilizationTimeout` specifies the timeout count value used when checking that the LPECO clock is stabilized.
 - 1 - 4294967295: Timeout count value used when checking that the LPECO clock is stabilized.

Note: *Even if `McuLpEcoAmpStabilizationTimeout` is deactivated, the LPECO clock status will be checked once.*

- `McuLpEcoFrequency` specifies the frequency of the LPECO clock oscillator (in Hz).

Note: *If `McuLpEcoEnable` is FALSE, this parameter must be set to 0.0 (in Hz).*

The MCU LPECO clock settings configuration holds the following container:

- `McuLpEcoPrescalerSettings` (see [MCU LPECO prescaler settings](#))

4.5.1.5 MCU LPECO prescaler settings

Use the following parameters to configure the LPECO prescaler:

- `McuLpEcoPrescalerEnable` enables or disables the LPECO prescaler.
- `McuLpEcoPrescalerValue` specifies the LPECO prescaler value.
 - 1 - 1024.99609375: LPECO prescaler value.
- `McuLpEcoPrescalerEnableTimeout` specifies the timeout count value used when checking that the LPECO prescaler is enabled.
 - 1 - 4294967295: Timeout count value used when checking that the LPECO prescaler is enabled.

Note: *Even if `McuLpEcoPrescalerEnableTimeout` is deactivated, the LPECO prescaler status will be checked once.*

- `McuLpEcoPrescaledFrequency` specifies the frequency of the prescaled LPECO clock (in Hz).

Note: *`McuLpEcoPrescaledFrequency` automatically displays the resulting frequency calculated by following formula: $McuLpEcoPrescaledFrequency = McuLpEcoFrequency / McuLpEcoPrescalerValue$*

4.5.1.6 MCU ILO clock settings

The `McuIloSettings` container has the following parameters to configure the ILO clocks:

- `McuIlo0Enable` enables or disables the ILO0 clock.
- `McuIlo0OnBackupEnable` enables or disables the ILO0 remaining on if backup domain is supported by the derivative.
- `McuIlo0MonitorEnable` enables or disables the internal ILO0 clock monitoring circuit.

Note: *This parameter must be set to FALSE as the ILO0 clock monitoring feature is no longer supported.*

- `McuIlo0Frequency` specifies the frequency of the ILO0 clock oscillator (in Hz).

Note: *If `McuIlo0Enable` is FALSE, this parameter must be set to 0.0 (in Hz).*

The MCU ILO clock settings configuration holds the following container:

- `McuIlo1Settings` (see [MCU ILO1 clock settings](#))

4.5.1.7 MCU ILO1 clock settings

The `McuIlo1Settings` container has the following parameters to configure the ILO1 clock:

- `McuIlo1Enable` enables or disables the ILO1 clock.
- `McuIlo1MonitorEnable` enables or disables the internal ILO1 clock monitoring circuit.

Note: *This parameter must be set to FALSE as the ILO1 clock monitoring feature is no longer supported.*

- `McuIlo1Frequency` specifies the frequency of the ILO1 clock oscillator (in Hz).

Note: *If `McuIlo1Enable` is FALSE, this parameter must be set to 0.0 (in Hz).*

4.5.1.8 MCU WCO clock settings

The `McuWcoSettings` container has the following parameters to configure the WCO clock:

- `McuWcoEnable` enables or disables the WCO clock.
- `McuWcoStopForUpdate` enables or disables to stop the WCO clock once before setting.

Note: *If `McuWcoStopForUpdate` is FALSE, setting of the WCO clock will be skipped when it is running.*

- `McuWcoType` specifies the type of board-level connections to the WCO pins.
 - `MCU_WCO_WATCH_CRYSTAL`: Watch crystal
 - `MCU_WCO_CLOCK_SIGNAL`: Clock signal
- `McuWcoPrescaler` specifies the prescaler for real time clock. This parameter can be set when `McuWcoEnable` is TRUE and `McuWcoType` is `MCU_WCO_CLOCK_SIGNAL`.
 - `MCU_WCO_SQUAREWAVE_32768HZ`: 32768 Hz square wave.
 - `MCU_WCO_SINEWAVE_60HZ`: 60 Hz sine wave.
 - `MCU_WCO_SINEWAVE_50HZ`: 50 Hz sine wave.
- `McuWcoStabilizationTimeout` specifies the timeout count value used when checking that the WCO clock is stabilized.
 - 1 - 4294967295: Timeout count value used when checking that the WCO clock is stabilized.

Note: *Even if `McuWcoStabilizationTimeout` is deactivated, the WCO clock status will be checked once.*

- `McuWcoFrequency` specifies the frequency of the WCO clock oscillator (in Hz).

Note: *If `McuWcoEnable` is FALSE, this parameter must be set to 0.0 (in Hz).*

4.5.2 MCU clock settings

The `McuClockSettings` container holds the configurations for clock common settings:

- `McuPclkEnableTimeout` specifies the timeout count value used when checking that the PCLK is enabled. This parameter is not used.
 - 1 - 4294967295: Timeout count value used when checking that the PCLK is enabled.
- `McuPeriGroupBusTransferTimeout` specifies the AHB-Lite bus transfer timeout value in peripheral group clock cycle.
 - 0 - 65534: AHB-Lite bus transfer timeout value.
- `McuBackupClockSource` specifies the source clock of the backup clock.
 - `MCU_CLOCK_WCO`: WCO clock.
 - `MCU_CLOCK_ALTBK`: Alternate backup domain clock (LF clock).
 - `MCU_CLOCK_ILO0`: ILO0 clock.
 - `MCU_CLOCK_LPECO_PRESCALE`: Prescaled LPECO.
- `McuBackupClockFrequency` is the frequency of the backup clock (in Hz).

Note: *`McuBackupClockFrequency` automatically displays the resulting frequency calculated by following formula: $McuBackupClockFrequency = \text{The frequency of the clock specified by } McuBackupClockSource$*

- `McuFast0ClockFrequency` is the frequency of the fast 0 clock (in Hz).

Note: *`McuFast0ClockFrequency` automatically displays the resulting frequency calculated by following formula:*

If the device supports Arm® Cortex®-M4 CPU, $McuFast0ClockFrequency = (The\ value\ of\ McuClockRootFrequency\ for\ which\ the\ corresponding\ McuClockRoot\ is\ set\ to\ MCU_CLOCK_ROOT0) / McuFast0ClockDivision$.

If the device supports Arm® Cortex®-M7 CPU, $McuFast0ClockFrequency = (The\ value\ of\ McuClockRootFrequency\ for\ which\ the\ corresponding\ McuClockRoot\ is\ set\ to\ MCU_CLOCK_ROOT1) / McuFast0ClockDivision$.

- `McuFast0ClockDivision` specifies the division value of the fast 0 clock.
 - 1.0 - 256.96875: Fast 0 clock division value.

Note: *Fractional value cannot be configured on some subderivatives.*

- `McuFast1ClockFrequency` is the frequency of the fast 1 clock (in Hz).

Note: *`McuFast1ClockFrequency` automatically displays the resulting frequency calculated by following formula: $McuFast1ClockFrequency = (The\ value\ of\ McuClockRootFrequency\ for\ which\ the\ corresponding\ McuClockRoot\ is\ set\ to\ MCU_CLOCK_ROOT1) / McuFast1ClockDivision$*

- `McuFast1ClockDivision` specifies the division value of the fast 1 clock.
 - 1.0 – 256.96875: Fast 1 clock division value.
- `McuSlowClockFrequency` is the frequency of the slow clock (in Hz).

Note: *`McuSlowClockFrequency` automatically displays the resulting frequency calculated by following formula:*

If the device supports Arm® Cortex®-M4 CPU, $McuSlowClockFrequency = McuPeriClockFrequency / McuSlowClockDivision$.

If the device supports Arm® Cortex®-M7 CPU, $McuSlowClockFrequency = McuMemClockFrequency / McuSlowClockDivision$.

- `McuSlowClockDivision` specifies the division value of the slow clock.
 - 1 - 256: Slow clock division value.
- `McuPeriClockFrequency` is the frequency of the peripheral clock (in Hz).

Note: *`McuPeriClockFrequency` automatically displays the resulting frequency calculated by following formula: $McuPeriClockFrequency = (The\ value\ of\ McuClockRootFrequency\ for\ which\ the\ corresponding\ McuClockRoot\ is\ set\ to\ MCU_CLOCK_ROOT0) / McuPeriClockDivision$*

- `McuPeriClockDivision` specifies the division value of the peripheral clock.
 - 1 - 256: Peripheral clock division value.
- `McuMemClockFrequency` is the frequency of the memory clock (in Hz).

Note: *`McuMemClockFrequency` automatically displays the resulting frequency calculated by following formula: $McuMemClockFrequency = (The\ value\ of\ McuClockRootFrequency\ for\ which\ the\ corresponding\ McuClockRoot\ is\ set\ to\ MCU_CLOCK_ROOT0) / McuMemClockDivision$*

- `McuMemClockDivision` specifies the division value of the memory clock.
 - 1 - 256: Memory clock division value.
- `McuTrcDbgClockFrequency` is the frequency of the trace and debug clock (in Hz).

Note: *`McuTrcDbgClockFrequency` automatically displays the resulting frequency calculated by following formula: $\text{McuTrcDbgClockFrequency} = (\text{The value of } \text{McuClockRootFrequency} \text{ for which the corresponding } \text{McuClockRoot} \text{ is set to } \text{MCU_CLOCK_ROOT0}) / \text{McuTrcDbgClockDivision}$*

- `McuTrcDbgClockDivision` specifies the division value of the trace and debug clock.
 - 1 - 256: Trace and debug clock division value.
- `McuFlashWaitCycle` specifies the wait cycle for accessing the FLASH memory.
 - 0 - 15: Wait cycle for accessing the FLASH memory.
- `McuFastRomWaitCycle` specifies the wait cycle for accessing the ROM on the fast clock domain.
 - 0 - 3: Wait cycle for accessing the ROM on the fast clock domain.
- `McuSlowRomWaitCycle` specifies the wait cycle for accessing the ROM on the slow clock domain.
 - 0 - 3: Wait cycle for accessing the ROM on the slow clock domain.
- `McuFastRam0WaitCycle` specifies the wait cycle for accessing the RAM0 on the fast clock domain.
 - 0 - 3: Wait cycle for accessing the RAM0 on the fast clock domain.
- `McuSlowRam0WaitCycle` specifies the wait cycle for accessing the RAM0 on the slow clock domain.
 - 0 - 3: Wait cycle for accessing the RAM0 on the slow clock domain.
- `McuFastRam1WaitCycle` specifies the wait cycle for accessing the RAM1 on the fast clock domain.
 - 0 - 3: Wait cycle for accessing the RAM1 on the fast clock domain.
- `McuSlowRam1WaitCycle` specifies the wait cycle for accessing the RAM1 on the slow clock domain.
 - 0 - 3: Wait cycle for accessing the RAM1 on the slow clock domain.
- `McuFastRam2WaitCycle` specifies the wait cycle for accessing the RAM2 on the fast clock domain.
 - 0 - 3: Wait cycle for accessing the RAM2 on the fast clock domain.
- `McuSlowRam2WaitCycle` specifies the wait cycle for accessing the RAM2 on the slow clock domain.
 - 0 - 3: Wait cycle for accessing the RAM2 on the slow clock domain.
- `McuCsvReferenceClock` specifies the reference clock of the clock supervisor.
 - `MCU_CLOCK_IMO`: IMO clock
 - `MCU_CLOCK_EXTCLK`: External clock
 - `MCU_CLOCK_ECO`: ECO clock
 - `MCU_CLOCK_ALTHF`: ALTHF clock

The MCU clock settings configuration holds the following containers:

- `McuClockPathSettings` (see [MCU clock path settings](#))
- `McuFllSettings` (see [MCU FLL clock settings](#))
- `McuPllSettings` (see [MCU PLL clock settings](#))
- `McuSscgPllSettings` (see [MCU SSCG PLL clock settings](#))
- `McuClockRootSettings` (see [MCU clock root settings](#))
- `McuPclkGroupSettings` (see [MCU PCLK group settings](#))
- `McuPeriGroupSettings` (see [MCU peripheral group settings](#))
- `McuLfclockSettings` (see [MCU LF clock settings](#))
- `McuPumpClockSettings` (see [MCU pump clock settings](#))

- `McuTimerClockSettings` (see [MCU timer clock settings](#))
- `McuClockOutputSettings` (see [MCU clock output settings](#))
- `McuCsvSettings` (see [MCU clock supervisor settings](#))

4.5.2.1 MCU clock path settings

The `McuClockPathSettings` container has the following parameters to configure the clock path:

- `McuClockPath` specifies the clock path.
 - `MCU_CLOCK_PATH<n>`: Clock path <n> (<n> = 0 ... 15).

Note: *In the same `McuClockSettingConfig` container, `McuClockPath` must be unique.*

Selectable clock paths depend on the subderivative. The clock path not used for FLL clock, PLL clock and SSCG PLL clock can be set.

- `McuClockPathFrequency` is the frequency of the clock path specified by `McuClockPath` (in Hz).

Note: *`McuClockPathFrequency` automatically displays the resulting frequency calculated by following formula: $McuClockPathFrequency = (The\ frequency\ of\ the\ clock\ specified\ by\ McuClockPathSource)$*

- `McuClockPathSource` specifies the source clock for the clock path specified by `McuClockPath`.
 - `MCU_CLOCK_IMO`: IMO clock
 - `MCU_CLOCK_EXTCLK`: External clock
 - `MCU_CLOCK_ECO`: ECO clock
 - `MCU_CLOCK_LPECO`: LPECO clock
 - `MCU_CLOCK_ILO0`: ILO0 clock
 - `MCU_CLOCK_ILO1`: ILO1 clock
 - `MCU_CLOCK_WCO`: WCO clock
 - `MCU_CLOCK_ALTHF`: ALTHF clock
 - `MCU_CLOCK_ALTLF`: ALTLF clock
 - `MCU_CLOCK_DSI<n>`: DSI output <n> clock (<n> = 0 ... 15).

Note: *Selectable source clocks depend on the subderivative.*

4.5.2.2 MCU FLL clock settings

The `McuFllSettings` container has the following parameters to configure the FLL clock:

- `McuFllEnable` enables or disables the FLL clock.

Note: *If this parameter is `TRUE`, `McuFllCcoEnable` must be set to `TRUE`.*

- `McuFllStopForUpdate` enables or disables to stop the FLL clock once before setting.

Note: *If `McuFllStopForUpdate` is `FALSE`, setting of the FLL clock will be skipped when it is running.*

- `McuFllAutoDistributeEnable` enables or disables the automatic distribution of the FLL clock.

Note: *If `McuFllEnable` is `TRUE`, this parameter should be set to `TRUE`.*

- `McuFllAutoDistributeType` specifies the automatic distribution type of the FLL clock.
 - `MCU_DISTRIBUTE_AFTER_LOCKED`: The FLL clock will be automatically distributed after being locked. If it is unlocked after being locked, it will be switched to its reference input clock automatically (bypass mode).
 - `MCU_DISTRIBUTE_ONLY_LOCKED`: The FLL clock will be automatically distributed after being locked. If it is unlocked after being locked, it will be gated OFF.

Note: *If `McuFllEnable` is TRUE, this parameter should be set to `MCU_DISTRIBUTE_AFTER_LOCKED`.*

- `McuFllStabilizationTimeout` specifies the timeout count value used when checking whether the FLL clock is stabilized.
 - 1 - 4294967295: Timeout count value used when checking whether the FLL clock is stabilized.
- `McuFllFrequency` is the frequency of the FLL clock (in Hz).

Note: *`McuFllFrequency` automatically displays the resulting frequency calculated by following formula: $McuFllFrequency = ((The\ frequency\ of\ the\ clock\ specified\ by\ McuFllSource / McuFllReferenceDivision) * McuFllMultiplication) / McuFllOutputDivision$*

- `McuFllSource` specifies the source clock of the FLL clock:
 - `MCU_CLOCK_IMO`: IMO clock
 - `MCU_CLOCK_EXTCLK`: External clock
 - `MCU_CLOCK_ECO`: ECO clock
 - `MCU_CLOCK_LPECO`: LPECO clock
 - `MCU_CLOCK_ILO0`: ILO0 clock
 - `MCU_CLOCK_ILO1`: ILO1 clock
 - `MCU_CLOCK_WCO`: WCO clock
 - `MCU_CLOCK_ALTHF`: ALTHF clock
 - `MCU_CLOCK_ALTLF`: ALTLF clock
 - `MCU_CLOCK_DSI<n>`: DSI output <n> clock (<n> = 0 ... 15)

Note: *Selectable source clocks depend on the subderivative.*

- `McuFllReferenceDivision` specifies the reference division value of the FLL clock.
 - 1 - 8191: FLL clock reference division value.
- `McuFllOutputDivision` specifies the output division value of the FLL clock.
 - 1 - 2: FLL clock output division value.
- `McuFllMultiplication` specifies the multiplication value of the FLL clock.
 - 0 - 262143: FLL clock multiplication value.
- `McuFllCcoEnable` enables or disables the CCO.
- `McuFllCcoOffset` specifies the allowed maximum value of the CCO offset.
 - 0 - 255: CCO offset allowed maximum value.
- `McuFllCcoAutoUpdateDisable` enables or disables the CCO frequency update by the FLL hardware.
- `McuFllCcoFrequencyCode` specifies the CCO frequency code.
 - 0 - 511: CCO frequency code.
- `McuFllCcoStabilizationTimeout` specifies the timeout count value used when checking whether the CCO is stabilized.
 - 1 - 4294967295: Timeout count value used when checking that the CCO is stabilized.

Note: *Even if `McuFllCcoStabilizationTimeout` is deactivated, the CCO status will be checked once.*

- `McuFllLockTolerance` specifies the lock tolerance, which is the error threshold when the FLL output is considered locked to the reference input.
 - 1 - 256: Lock tolerance value.
- `McuFllUpdateTolerance` specifies the update tolerance, which is the error threshold for when the FLL will update the CCO frequency settings.
 - 0 - 254: Update tolerance value.
- `McuFllSettlingCount` specifies the number of undivided reference clock cycles to wait after changing the CCO trim until the loop measurement restarts.
 - 0 - 8191: Reference clock cycle.
- `McuFllLoopFilterIGain` specifies the FLL loop filter integral gain setting.
 - `MCU_FLL_LOOP_FILTER_GAIN_1_BY_256`: 1/256
 - `MCU_FLL_LOOP_FILTER_GAIN_1_BY_128`: 1/128
- `McuFllLoopFilterPGain` specifies the FLL loop filter proportional gain setting.
 - `MCU_FLL_LOOP_FILTER_GAIN_1_BY_256`: 1/256
 - `MCU_FLL_LOOP_FILTER_GAIN_1_BY_128`: 1/128

4.5.2.3 MCU PLL clock settings

The `McuPllSettings` container has the following parameters to configure the PLL clock:

- `McuPllType` specifies the PLL clock.
 - `MCU_CLOCK_PLL<n>`: PLL<n> clock (<n> = 0 ... 14).

Note: *In the same `McuClockSettingConfig` container, `McuPllType` must be unique.*

Selectable PLL clocks depend on the subderivative.

- `McuPllEnable` enables or disables the PLL clock specified by `McuPllType`.
- `McuPllStopForUpdate` enables or disables to stop PLL clock specified by `McuPllType` once before setting.

Note: *If `McuPllStopForUpdate` is FALSE, setting the PLL clock specified by `McuPllType` will be skipped when it is running.*

- `McuPllAutoDistributeEnable` enables or disables the automatic distribution of the PLL clock specified by `McuPllType`.

Note: *If `McuPllAutoDistributeEnable` is TRUE, the PLL clock specified by `McuPllType` will be automatically distributed after locked and the manual distribution process in `Mcu_DistributePllClock()` will be skipped.*

- `McuPllAutoDistributeType` specifies the automatic distribution type of the PLL clock specified by `McuPllType`.
 - `MCU_DISTRIBUTE_AFTER_LOCKED`: The PLL clock specified by `McuPllType` will be automatically distributed after locked. If it becomes unlocked after locked, it will be automatically switched to its reference input clock (bypass mode).
 - `MCU_DISTRIBUTE_ONLY_LOCKED`: The PLL clock specified by `McuPllType` will be automatically distributed after locked. If it becomes unlocked after locked, it will be gated OFF.

- `McuPllStabilizationTimeout` specifies the timeout count value used when checking the PLL clock specified by `McuPllType` is stabilized.

1 - 4294967295: Timeout count value used when checking that the PLL clock is stabilized.

- `McuPllFrequency` is the frequency of the PLL clock (in Hz).

Note: *`McuPllFrequency` automatically displays the resulting frequency calculated by following formula: $McuPllFrequency = ((The\ frequency\ of\ the\ clock\ specified\ by\ McuPllSource / McuPllReferenceDivision) * McuPllFeedbackDivision) / McuPllOutputDivision$*

- `McuPllSource` specifies the source clock of the PLL clock specified by `McuPllType`.
 - `MCU_CLOCK_IMO`: IMO clock.
 - `MCU_CLOCK_EXTCLK`: External clock.
 - `MCU_CLOCK_ECO`: ECO clock.
 - `MCU_CLOCK_LPECO`: LPECO clock.
 - `MCU_CLOCK_ILO0`: ILO0 clock.
 - `MCU_CLOCK_ILO1`: ILO1 clock.
 - `MCU_CLOCK_WCO`: WCO clock.
 - `MCU_CLOCK_ALTHF`: ALTHF clock.
 - `MCU_CLOCK_ALTLF`: ALTLF clock.
 - `MCU_CLOCK_DSI<n>`: DSI output <n> clock (<n> = 0 ... 15).

Note: *Selectable source clocks depend on the subderivative.*

- `McuPllReferenceDivision` specifies the reference division value of the PLL clock specified by `McuPllType`.
 - 1 - 20: PLL clock reference division value.
- `McuPllOutputDivision` specifies the output division value of the PLL clock specified by `McuPllType`.
 - 2 - 16: PLL clock output division value.
- `McuPllFeedbackDivision` specifies the feedback division value of the PLL clock specified by `McuPllType`.
 - 22 - 112: PLL clock feedback division value.
- `McuPllLockSensitivity` specifies the sensitivity of the lock detection of the PLL clock specified by `McuPllType`.
 - `MCU_LOCK_SENSITIVITY_NORMAL`: Normal sensitivity.
 - `MCU_LOCK_SENSITIVITY_REDUCED`: Reduced sensitivity.

4.5.2.4 MCU SSCG PLL clock settings

The `McuSscgPllSettings` container has the following parameters to configure the SSCG PLL clock:

- `McuSscgPllType` specifies the SSCG PLL clock.
 - `MCU_CLOCK_SSCG_PLL<n>`: SSCG PLL<n> clock (<n> = 0 ... 14).

Note: *In the same `McuClockSettingConfig` container, `McuSscgPllType` must be unique.*

Selectable SSCG PLL clock depend on the subderivative.

- `McuSscgPllEnable` enables or disables the SSCG PLL clock specified by `McuSscgPllType`.

- `McuSscgPllStopForUpdate` enables or disables to stop SSCG PLL clock specified by `McuSscgPllType` once before setting.

Note: *If `McuSscgPllStopForUpdate` is FALSE, setting the SSCG PLL clock specified by `McuSscgPllType` will be skipped when it is running.*

- `McuSscgPllAutoDistributeEnable` enables or disables the automatic distribution of the SSCG PLL clock specified by `McuSscgPllType`.

Note: *If `McuSscgPllAutoDistributeEnable` is TRUE, the SSCG PLL clock specified by `McuSscgPllType` will be automatically distributed after locked and the manual distribution process in `Mcu_DistributePllClock()` will be skipped.*

- `McuSscgPllAutoDistributeType` specifies the automatic distribution type of the SSCG PLL clock specified by `McuPllType`.
 - `MCU_DISTRIBUTE_AFTER_LOCKED`: The SSCG PLL clock specified by `McuSscgPllType` will be automatically distributed after locked. If it becomes unlocked after locked, it will be automatically switched to its reference input clock (bypass mode).
 - `MCU_DISTRIBUTE_ONLY_LOCKED`: The SSCG PLL clock specified by `McuSscgPllType` will be automatically distributed after locked. If it becomes unlocked after locked, it will be gated off.
- `McuSscgPllStabilizationTimeout` specifies the timeout count value used when checking the SSCG PLL clock specified by `McuSscgPllType` is stabilized.

1 - 4294967295: Timeout count value used when checking that the SSCG PLL clock is stabilized.

- `McuSscgPllFrequency` is the frequency of the SSCG PLL clock (in Hz).

Note: *`McuSscgPllFrequency` automatically displays the resulting frequency calculated by following formula:*

$$\text{McuSscgPllFrequency} = ((\text{The frequency of the clock specified by } \text{McuSscgPllSource} / \text{McuSscgPllReferenceDivision}) * \text{McuSscgPllFeedbackDivision}) / \text{McuSscgPllOutputDivision}$$

Note: *If `McuSscgPllModulationEnable` is TRUE, `McuSscgPllFrequency` displays the average of the modulated frequencies.*

- `McuSscgPllSource` specifies the source clock of the SSCG PLL clock specified by `McuSscgPllType`.
 - `MCU_CLOCK_IMO`: IMO clock.
 - `MCU_CLOCK_EXTCLK`: External clock.
 - `MCU_CLOCK_ECO`: ECO clock.
 - `MCU_CLOCK_LPECO`: LPECO clock.
 - `MCU_CLOCK_ILO0`: ILO0 clock.
 - `MCU_CLOCK_ILO1`: ILO1 clock.
 - `MCU_CLOCK_WCO`: WCO clock.
 - `MCU_CLOCK_ALTHF`: ALTHF clock.
 - `MCU_CLOCK_ALTLF`: ALTLF clock.
 - `MCU_CLOCK_DSI<n>`: DSI output <n> clock (<n> = 0 ... 15).

Note: *Selectable source clocks depend on the subderivative.*

- `McuSscgPllReferenceDivision` specifies the reference division value of the SSCG PLL clock specified by `McuSscgPllType`.
 - 1 - 16: SSCG PLL clock reference division value.
- `McuSscgPllOutputDivision` specifies the output division value of the SSCG PLL clock specified by `McuSscgPllType`.
 - 2 - 16: SSCG PLL clock output division value.
- `McuSscgPllFeedbackDivision` specifies the feedback division value of the SSCG PLL clock specified by `McuSscgPllType`.
 - 16.0 - 200.999999940395355: SSCG PLL clock feedback division value.
- `McuSscgPllLockSensitivity` specifies the sensitivity of the lock detection of the SSCG PLL clock specified by `McuSscgPllType`.
 - `MCU_LOCK_SENSITIVITY_INTEGER`: Integer divider mode without spreading.
 - `MCU_LOCK_SENSITIVITY_FRACTIONAL_OR_SPREADING`: Fractional divider mode or spreading mode.
- `McuSscgPllFractionalDivisionEnable` enables or disables the fractional feedback division of the SSCG PLL clock specified by `McuSscgPllType`.
- `McuSscgPllFractionalDivisionDitheringEnable` enables or disables the dithering for the fractional feedback division of the SSCG PLL clock specified by `McuSscgPllType`.
- `McuSscgPllModulationEnable` enables or disables the SSCG modulation of the SSCG PLL clock specified by `McuSscgPllType`.
- `McuSscgPllModulationMode` specifies the SSCG modulation mode of the SSCG PLL clock specified by `McuSscgPllType`.
 - `MCU_SSCG_MODE_DOWN_SPREAD`: Down spread mode.
- `McuSscgPllModulationDepth` specifies the SSCG modulation depth of the SSCG PLL clock specified by `McuSscgPllType` as a percentage of the non-modulated clock.
 - `MCU_SSCG_DEPTH_0_5_PERCENT`: -0.5% for down spread mode.
 - `MCU_SSCG_DEPTH_1_0_PERCENT`: -1.0% for down spread mode.
 - `MCU_SSCG_DEPTH_2_0_PERCENT`: -2.0% for down spread mode.
 - `MCU_SSCG_DEPTH_3_0_PERCENT`: -3.0% for down spread mode.
- `McuSscgPllModulationRate` specifies the SSCG modulation rate of the SSCG PLL clock specified by `McuSscgPllType`.
 - `MCU_SSCG_RATE_FPFY_BY_4096`: Modulation rate is fPFY / 4096.
 - `MCU_SSCG_RATE_FPFY_BY_2048`: Modulation rate is fPFY / 2048.
 - `MCU_SSCG_RATE_FPFY_BY_1024`: Modulation rate is fPFY / 1024.
 - `MCU_SSCG_RATE_FPFY_BY_512`: Modulation rate is fPFY / 512.
 - `MCU_SSCG_RATE_FPFY_BY_256`: Modulation rate is fPFY / 256.

Note: *Configuring `MCU_SSCG_RATE_FPFY_BY_256` is possible only when fPFY is 8 MHz.*

- `McuSscgPllModulationDitheringEnable` enables or disables the dithering for the SSCG modulation of the SSCG PLL clock specified by `McuSscgPllType`.

Note: *`McuSscgPllModulationDitheringEnable` is not supported and is always disabled.*

4.5.2.5 MCU clock root settings

`McuClockRootSettings` container has the following parameters to configure the clock root:

- `McuClockRoot` specifies the clock root.
 - `MCU_CLOCK_ROOT<n>`: clock root <n> (<n> = 0 ... 15).

Note: *In the same `McuClockSettingConfig` container, `McuClockRoot` must be unique.*

Selectable clock roots depend on the subderivative.

- `McuClockRootEnable` enables or disables the clock root specified by `McuClockRoot`.

Note: *If `McuClockRoot` is `MCU_CLOCK_ROOT0`, `McuClockRootEnable` must be set to `TRUE`.*

- `McuClockRootFrequency` is the frequency of the clock root specified by `McuClockRoot` (in Hz).

Note: *`McuClockRootFrequency` automatically displays the resulting frequency calculated by following formula: $McuClockRootFrequency = (The\ frequency\ of\ the\ clock\ specified\ by\ McuClockRootSource) / McuClockRootDivision$*

- `McuClockRootSource` specifies the source clock of the current clock root.
 - `MCU_CLOCK_FLL`: FLL clock.
 - `MCU_CLOCK_SSCG_PLL`: SSCG PLL clock.
 - `MCU_CLOCK_PLL`: PLL clock.
 - `MCU_CLOCK_PATH`: Clock path.
 - `MCU_CLOCK_IMO`: IMO clock
- `McuClockRootSscgPllRef` selects the SSCG PLL clock from `McuSscgPllSettings` to refer as the source clock of the clock root specified by `McuClockRoot`.

Note: *This parameter is available only if `McuClockRootSource` is `MCU_CLOCK_SSCG_PLL`.*

- `McuClockRootPllRef` selects the PLL clock from `McuPllSettings` to refer as the source clock of the clock root specified by `McuClockRoot`.

Note: *This parameter is available only if `McuClockRootSource` is `MCU_CLOCK_PLL`.*

- `McuClockRootPathRef` selects the clock path from `McuClockPathSettings` to refer as the source clock of the clock root specified by `McuClockRoot`.

Note: *This parameter is available only if `McuClockRootSource` is `MCU_CLOCK_PATH`.*

- `McuClockRootDivision` specifies the division value of the clock root specified by `McuClockRoot`.
 - `MCU_CLK_DIV_1`: Divided by 1.
 - `MCU_CLK_DIV_2`: Divided by 2.
 - `MCU_CLK_DIV_4`: Divided by 4.
 - `MCU_CLK_DIV_8`: Divided by 8.

4.5.2.6 MCU PCLK group settings

The `McuPclkGroupSettings` container has the following parameters to configure the PCLK group:

- `McuPclkGroup` specifies the PCLK group.
 - `MCU_PCLK_GROUP<n>`: PCLK group <n> (<n> = 0 ... 15).

Note: *Selectable PCLK groups depend on the subderivative.
In the same `McuClockSettingConfig` container, `McuPclkGroup` must be unique*

The `Mcu PCLK` group settings configuration holds the following containers:

- `McuPclkDividerSettings` (see [MCU PCLK divider settings](#))
- `McuPclkSettings` (see [MCU PCLK settings](#))

4.5.2.7 MCU PCLK divider settings

`McuPclkDividerSettings` container has the following parameters to configure the PCLK divider:

- `McuPclkDividerType` specifies the PCLK divider type.
 - `MCU_PCLK_DIVIDER_8`: 8.0 clock divider.
 - `MCU_PCLK_DIVIDER_16`: 16.0 clock divider.
 - `MCU_PCLK_DIVIDER_16_5`: 16.5 clock divider.
 - `MCU_PCLK_DIVIDER_24_5`: 24.5 clock divider.

Note: *Selectable PCLK dividers depend on the subderivative.*

- `McuPclkDividerIndex` specifies the index of the PCLK divider specified by `McuPclkDividerType`.

Note: *In the same `McuClockSettingConfig` container, `McuPclkDividerIndex` must be unique for each PCLK divider type.*

- `McuPclkDividerEnable` enables or disables the PCLK divider.
- `McuPclkDividerStopForUpdate` stops an already running PCLK divider, once, specified by `McuPclkDividerType` and `McuPclkDividerIndex` before setting the clock.

Note: *If `McuPclkDividerStopForUpdate` is FALSE, setting the PCLK divider will be skipped when it is running.*

- `McuPclkDividerValue` specifies the division value of the PCLK divider specified by `McuPclkDividerType` and `McuPclkDividerIndex`.

Configurable division value depends on the `McuPclkDividerType`.

- 1 - 256: In case of `MCU_PCLK_DIVIDER_8`.
- 1 - 65536: In case of `MCU_PCLK_DIVIDER_16`.
- 1 - 65536.96875: In case of `MCU_PCLK_DIVIDER_16_5`.
- 1 - 16777216.96875: In case of `MCU_PCLK_DIVIDER_24_5`.

Note: *If `McuPclkDividerType` is `MCU_PCLK_DIVIDER_8` or `MCU_PCLK_DIVIDER_16`, this parameter must be an integer value.*

If `McuPclkDividerType` is `MCU_PCLK_DIVIDER_16_5` or `MCU_PCLK_DIVIDER_24_5`, the value after the decimal point of this parameter must be five digits or less.

If the fractional part of this parameter is not a multiple of $1/32$, the value obtained by dividing it by $1/32$ is truncated and set to the hardware register. For example, if the value after the decimal point of this parameter is 0.96874, the value obtained by dividing it by $1/32$ will be 30.99968 and then the value 30 is set to the hardware register.

- `McuPclkPhaseAlignDividerRef` selects PCLK divider from `McuPclkDividerSettings` to reference for phase alignment.

Note: *If `McuPclkPhaseAlignDividerRef` is deactivated, the PCLK divider specified by `McuPclkDividerType` and `McuPclkDividerIndex` will be aligned with peripheral clock.*

The `McuPclkDividerSettings` preceding the current one must be selected.

4.5.2.8 MCU PCLK settings

`McuPclkSettings` container has the following parameters to configure the PCLK:

- `McuPclk` specifies the PCLK.
 - `MCU_PCLK_CPUSSE_CLOCK_TRACE_IN`: Trace clock.
 - `MCU_PCLK_SMARTIO0_CLOCK`: SMART IO #0.

Note: *Selectable PCLKs depend on the subderivative.*

In the same `McuPclkGroupSettings` container, `McuPclk` must be unique.

- `McuPclkEnable` enables or disables the PCLK clock specified by `McuPclk`.
- `McuPclkFrequency` is the frequency of the PCLK specified by `McuPclk` (in Hz).

Note: *`McuPclkFrequency` automatically displays the resulting frequency calculated by following formula:*

If PCLK divider is in PCLK group 0, $McuPclkFrequency = McuPeriClockFrequency / (McuPclkDividerValue \text{ of the PCLK divider selected by } McuPclkDividerRef)$

If PCLK divider is in PCLK group 1, $McuPclkFrequency = (The \text{ value of } McuClockRootFrequency \text{ for which the corresponding } McuClockRoot \text{ is set to } MCU_CLOCK_ROOT2) / (McuPclkDividerValue \text{ of the PCLK divider selected by } McuPclkDividerRef)$

- `McuPclkDividerRef` selects PCLK divider from `McuPclkDividerSettings` to refer as the divider of PCLK specified by `McuPclk`.

4.5.2.9 MCU peripheral group settings

The `McuPeriGroupSettings` container has the following parameters to configure the peripheral group:

- `McuPeriGroup` specifies the peripheral group.
 - `MCU_PERI_GROUP<n>_<peripheral group name>`: Peripheral group <n> (<n> = 0 ... 15).

Note: *Selectable peripheral groups depend on the subderivative.*

In the same `McuClockSettingConfig` container, `McuPeriGroup` must be unique

- `McuPeriGroupClockFrequency` is the frequency of the peripheral group clock specified by `McuPeriGroup` (in Hz).

Note: *`McuPeriGroupClockFrequency` automatically displays the resulting frequency calculated by following formula:*

If `McuPeriGroup` starts with `MCU_PERI_GROUP0_`, `MCU_PERI_GROUP1_`, or `MCU_PERI_GROUP2_`, then $McuPeriGroupClockFrequency = McuSlowClockFrequency$.

If `McuPeriGroup` starts with groups other than above, i.e.: `MCU_PERI_GROUP3_`, `MCU_PERI_GROUP4_`, or `MCU_PERI_GROUP5_` and so on, then $McuPeriGroupClockFrequency = McuPeriClockFrequency / McuPeriGroupClockDivision$ or $McuPeriGroupClockFrequency = (The\ value\ of\ McuClockRootFrequency\ for\ which\ the\ corresponding\ McuClockRoot\ is\ set\ to\ MCU_CLOCK_ROOT2) / McuPeriGroupClockDivision$.

- `McuPeriGroupClockDivision` specifies the division value of the peripheral group clock specified by `McuPeriGroup`.
 - 1 - 20: Peripheral group clock division value.

Mcu peripheral group settings configuration holds the following containers:

- `McuPeriGroupSlaveSettings` (see [MCU peripheral group slave settings](#))

4.5.2.10 MCU peripheral group slave settings

The `McuPeriGroupSlaveSettings` container has the following parameters to configure the slave of the peripheral group:

- `McuPeriGroupSlaveName`
 - `MCU_PERI_GROUP<n>_SLAVE<m>_<peripheral group slave name>`: Slave <m> of the peripheral group <n> (<n> = 0 ... 15, <m> = 0 ... 15).

Note: *Selectable peripheral group slaves depend on the subderivative.*

In the same `McuPeriGroupSettings` container, `McuPeriGroupSlaveName` must be unique

- `McuPeriGroupSlaveEnable` enables or disables the slave of the peripheral group specified by `McuPeriGroupSlaveName`.

Note: *If `McuPeriGroupSlaveName` starts with `MCU_PERI_GROUP0_SLAVE0_` or `MCU_PERI_GROUP0_SLAVE1_`, `McuPeriGroupSlaveEnable` must be set to `TRUE`. Also, if `McuPeriGroupSlaveName` starts with `MCU_PERI_GROUP0_SLAVE2_`, then `McuPeriGroupSlaveEnable` must be set to `TRUE` if the device supports Arm® Cortex®-M7 CPU.*

4.5.2.11 MCU LF clock settings

`McuLfClockSettings` container has the following parameters to configure the LF clock:

- `McuLfClockFrequency` is the frequency of the LF clock (in Hz).

Note: *`McuLfClockFrequency` automatically displays the resulting frequency calculated by following formula: $McuLfClockFrequency = (The\ frequency\ of\ the\ clock\ specified\ by\ McuLfClockSource)$*

- `McuLFClockSource` specifies the source clock of the LF clock.
 - `MCU_CLOCK_ILO0`: ILO0 clock.
 - `MCU_CLOCK_ILO1`: ILO1 clock.
 - `MCU_CLOCK_ECO_PRESCALE`: Prescaled ECO clock.
 - `MCU_CLOCK_LPECO_PRESCALE`: Prescaled LPECO clock.
 - `MCU_CLOCK_WCO`: WCO clock.
 - `MCU_CLOCK_ALTLF`: ALTLF clock.

Note: *`MCU_CLOCK_ECO_PRESCALE` must not be set to `McuLFClockSource` when the configuration is used for DeepSleep mode.*

4.5.2.12 MCU pump clock settings

The `McuPumpClockSettings` container has the following parameters to configure the pump clock:

Note: *`McuPumpClockSettings` is not supported and is always disabled.*

- `McuPumpClockEnable` enables or disables the pump clock.
- `McuPumpClockStopForUpdate` stops a running the pump clock before setting the clock.

Note: *If `McuPumpClockStopForUpdate` is FALSE, setting the pump clock will be skipped when it is running.*

- `McuPumpClockFrequency` is the frequency of the pump clock (in Hz).

Note: *`McuPumpClockFrequency` automatically displays the resulting frequency calculated by following formula: $\text{McuPumpClockFrequency} = \text{The frequency of the clock specified by } \text{McuPumpClockSource} / \text{McuPumpClockDivision}$.*

- `McuPumpClockSource` specifies the source clock of the pump clock.
 - `MCU_CLOCK_FLL`: FLL clock.
 - `MCU_CLOCK_SSCG_PLL`: SSCG PLL clock.
 - `MCU_CLOCK_PLL`: PLL clock.
 - `MCU_CLOCK_PATH`: Clock path.
- `McuPumpClockSscgPllRef` selects the SSCG PLL clock from `McuSscgPllSettings` as the source clock of the pump clock.
- `McuPumpClockPllRef` selects the PLL clock from `McuPllSettings` as the source clock of the pump clock.
- `McuPumpClockPathRef` selects the clock path from `McuClockPathSettings` as the source clock of the pump clock.
- `McuPumpClockDivision` specifies the division value of the pump clock.
 - `MCU_CLK_DIV_1`: Divided by 1.
 - `MCU_CLK_DIV_2`: Divided by 2.
 - `MCU_CLK_DIV_4`: Divided by 4.
 - `MCU_CLK_DIV_8`: Divided by 8.
 - `MCU_CLK_DIV_16`: Divided by 16.

4.5.2.13 MCU timer clock settings

McuTimerClockSettings container has the following parameters to configure the timer clock:

- McuTimerClockEnable enables or disables the timer clock.
- McuTimerClockStopForUpdate stops the running timer clock before setting the clock.

Note: *If McuTimerClockStopForUpdate is FALSE, setting the timer clock will be skipped when it is running.*

- McuTimerClockFrequency is the frequency of the timer clock (in Hz).

Note: *McuTimerClockFrequency automatically displays the resulting frequency calculated by following formula:*

If McuTimerClockSource is MCU_CLOCK_IMO, then McuTimerClockFrequency = McuImoFrequency / McuTimerClockDivision.

If McuTimerClockSource is MCU_CLOCK_HF0DIV, then McuTimerClockFrequency = ((The value of McuClockRootFrequency for which the corresponding McuClockRoot is set to MCU_CLOCK_ROOT0) / McuTimerClockInputDivision) / McuTimerClockDivision.

- McuTimerClockSource specifies the source clock of the timer clock.
 - MCU_CLOCK_HF0DIV: HF0 (clock root 0) clock divided by McuTimerClockInputDivision.
 - MCU_CLOCK_IMO: IMO clock.
- McuTimerClockInputDivision specifies the HF0 clock division value for the source clock of the timer clock.
 - MCU_CLK_DIV_1: Divided by 1.
 - MCU_CLK_DIV_2: Divided by 2.
 - MCU_CLK_DIV_4: Divided by 4.
 - MCU_CLK_DIV_8: Divided by 8.

Note: *This parameter is available only if McuTimerClockSource is MCU_CLOCK_HF0DIV.*

- McuTimerClockDivision specifies the division value of the timer clock.
 - 1 - 256: Timer clock division value.

4.5.2.14 MCU clock output settings

The McuClockOutputSettings container has the following parameters to configure the clock output:

- McuClockOutput0Enable enables or disables the clock output 0.

Note: *Because the clock output function enabled by McuClockOutput0Enable is for testing purposes only, McuClockOutput0Enable must not be set TRUE for production.*

Note: *A warning message will be reported if McuClockOutput0Enable is TRUE. This message indicates that the configuration in PORT module for the port pin used by the clock output 0 function will be ignored.*

- `McuClockOutput0Frequency` is the frequency of the clock output 0 (in Hz).

Note: *`McuClockOutput0Frequency` automatically displays the resulting frequency calculated by following formula. $McuClockOutput0Frequency = (The\ frequency\ of\ the\ clock\ specified\ by\ McuClockOutput0Source) / McuClockOutput0Division$.*

- `McuClockOutput0Source` specifies the source clock of the clock output 0.
 - `MCU_CLOCK_LOW`: Disabled and output is fixed low.
 - `MCU_CLOCK_ECO`: ECO clock.

Note: *Selectable source clocks depend on the subderivative.*

- `McuClockOutput0Division` specifies the division value of the clock output 0.
 - `MCU_CLK_DIV_1`: Divided by 1.
 - `MCU_CLK_DIV_2`: Divided by 2.
 - `MCU_CLK_DIV_4`: Divided by 4.
 - `MCU_CLK_DIV_8`: Divided by 8.
- `McuClockOutput1Enable` enables or disables the clock output 1.

Note: *Because the clock output function enabled by `McuClockOutput1Enable` is for testing purposes only, `McuClockOutput1Enable` must not be set `TRUE` for production.*

Note: *A warning message will be reported if `McuClockOutput1Enable` is `TRUE`. This message indicates that the configuration in `PORT` module for the port pin used by the clock output 1 function will be ignored.*

- `McuClockOutput1Frequency` is the frequency of the clock output 1 (in Hz).

Note: *`McuClockOutput1Frequency` automatically displays the resulting frequency calculated by following formula. $McuClockOutput1Frequency = (The\ frequency\ of\ the\ clock\ specified\ by\ McuClockOutput1Source) / McuClockOutput1Division$.*

- `McuClockOutput1Source` specifies the source clock of the clock output 1.
 - `MCU_CLOCK_LOW`: Disabled and output is fixed LOW.
 - `MCU_CLOCK_ECO`: ECO clock.

Note: *Selectable source clocks depend on the subderivative.*

- `McuClockOutput1Division` specifies the division value of the clock output 1.
 - `MCU_CLK_DIV_1`: Divided by 1.
 - `MCU_CLK_DIV_2`: Divided by 2.
 - `MCU_CLK_DIV_4`: Divided by 4.
 - `MCU_CLK_DIV_8`: Divided by 8.

4.5.2.15 MCU clock supervisor settings

The `McuCsvSettings` container has the following parameters to configure the clock supervisor:

- `McuCsvClock` specifies the monitoring clock of the clock supervisor.
 - `MCU_CLOCK_CSVREF`: Reference clock of the clock supervisor.
 - `MCU_CLOCK_LF`: LF clock.
 - `MCU_CLOCK_ILO0`: ILO0 clock.
 - `MCU_CLOCK_BACKUP`: Backup clock.
 - `MCU_CLOCK_ROOT<n>`: clock root <n> (<n> = 0 ... 15).

Note: *Selectable monitoring clocks depend on the subderivative.*

Note: *In the same `McuClockSettingConfig` container, `McuCsvClock` must be unique.*

- `McuCsvEnable` enables or disables the clock supervisor specified by `McuCsvClock`.

Note: *If this parameter is `TRUE`, monitoring clock and reference clock must be enabled.*

- `McuCsvPeriod` specifies the number of monitored clock cycles within a period.
 - 1 - 256: In case of `MCU_CLOCK_LF` and `MCU_CLOCK_ILO0`.
 - 1 - 65536: In case of `MCU_CLOCK_CSVREF` and `MCU_CLOCK_ROOT<n>`.
- `McuCsvStartupDelay` specifies the startup delay of the clock supervisor in reference clock cycles.
 - 1 - 256: In case of `MCU_CLOCK_LF` and `MCU_CLOCK_ILO0`.
 - 1 - 512: In case of `MCU_CLOCK_LF` and `MCU_CLOCK_ILO0` and `MCU_CLOCK_BACKUP`.
 - 1 - 65536: In case of `MCU_CLOCK_CSVREF` and `MCU_CLOCK_ROOT<n>`.

Note: *The valid range of `MCU_CLOCK_LF` and `MCU_CLOCK_ILO0` is device-specific. See the hardware register technical reference.*

- `McuCsvLowerLimit` specifies the lower limit of the clock supervisor in reference clock cycles.
 - 1 - 256: In case of `MCU_CLOCK_LF` and `MCU_CLOCK_ILO0`.
 - 1 - 65536: In case of `MCU_CLOCK_CSVREF` and `MCU_CLOCK_ROOT<n>`.

Note: *`McuCsvLowerLimit` must be less than `McuCsvUpperLimit` - 1.*

- `McuCsvUpperLimit` specifies the upper limit of the clock supervisor in reference clock cycles.
 - 1 - 256: In case of `MCU_CLOCK_LF` and `MCU_CLOCK_ILO0`.
 - 1 - 65536: In case of `MCU_CLOCK_CSVREF` and `MCU_CLOCK_ROOT<n>`.
- `McuCsvAction` specifies the action executed when the error is detected by the clock supervisor specified by `McuCsvClock`.
 - `MCU_CSV_ACTION_FAULT`: Fault report.
 - `MCU_CSV_ACTION_RESET`: Reset.

Note: *When `MCU_CSV_ACTION_FAULT` is configured, you should handle the fault report of the clock supervisor.*

4.5.3 MCU clock reference point

`McuClockReferencePoint` container has the following parameters to configure the clock references.

- `McuClock` selects the clock type for this clock reference point.
 - `MCU_CLOCK_IMO`: IMO clock.
 - `MCU_CLOCK_ECO`: ECO clock.

Note: *Selectable clocks depend on the subderivative.*

- `McuClockReferencePointFrequency` specifies the clock frequency of the selected by `McuClock` (in Hz). It is referenced by other modules.
- `McuClockReferencePointFrequency` will display the resulting frequency. These settings are evaluated and displayed in the resulting clock frequencies. This value will be assigned to the following definitions:
 - The definitions derived from the `McuModuleConfiguration` container short name, the `McuClockSettingConfig` container short name, `McuClockReferencePoint` container short name, and the `McuClock` parameter value are concatenated with "_" and prefixed with "MCU_".

Example:

`MCU_McuModuleConfiguration_0_McuClockSettingConfig_0_McuClockReferencePoint_0_MCU_CLOCK_IMO`.

4.6 MCU mode settings configuration

The `McuModeSettingConf` container has the following parameters to configure the mode settings:

- `McuMode` is a logical ID of the mode setting. This value will be assigned to the following symbolic names:
 - The symbolic name derived from the `McuModeSettingConf` container short name is prefixed with "`McuConf_McuModeSettingConf_`".

Example:

`McuConf_McuModeSettingConf_McuModeSettingConf_0`.

Note: *In the same `McuModuleConfiguration` container, `McuMode` must be unique and consecutive.*

- `McuTargetCpu` specifies the CPU which applies the mode specified by `McuCpuPowerMode`.
 - `MCU_CPU_CM0P`: Arm® Cortex®-M0+ CPU
 - `MCU_CPU_CM4`: Arm® Cortex®-M4 CPU
 - `MCU_CPU_CM7_0`: Arm® Cortex®-M7 CPU 0
 - `MCU_CPU_CM7_1`: Arm® Cortex®-M7 CPU 1

Note: *The mode setting must be applied on the CPU specified by `McuTargetCpu`.*

- `McuCpuPowerMode` specifies CPU power mode.
 - `MCU_CPUMODE_ACTIVE`: CPU Active mode.
 - `MCU_CPUMODE_SLEEP`: CPU Sleep mode.
 - `MCU_CPUMODE_DEEPSLEEP`: CPU DeepSleep mode.
 - `MCU_CPUMODE_HIBERNATE`: System Hibernate mode

Note: *To set to low-power mode, it is necessary to set all cores to Sleep or DeepSleep mode.*

- `McuEnableLowPowerTransition` specifies whether enter the low-power state or not.

- `McuMainCore0PowerMode` specifies the power mode of the Main Core 0 CPU power domain.
 - `MCU_POWERMODE_ENABLED`: Switch ON.
 - `MCU_POWERMODE_OFF`: Switch OFF.
 - `MCU_POWERMODE_RESET`: Reset.
 - `MCU_POWERMODE_RETAINED`: Put in retained mode.

Note: *This parameter is available only if `McuTargetCpu` is `MCU_CPU_CM0P`.*

Note: *`MCU_POWERMODE_RETAINED` can be effective only when Main Core 0 is in CPU DeepSleep mode.*

- `McuMainCore1PowerMode` specifies the power mode of the Main Core 1 CPU power domain.
 - `MCU_POWERMODE_ENABLED`: Switch ON.
 - `MCU_POWERMODE_OFF`: Switch OFF.
 - `MCU_POWERMODE_RESET`: Reset.
 - `MCU_POWERMODE_RETAINED`: Put in retained mode.

Note: *This parameter is available only if `McuTargetCpu` is `MCU_CPU_CM0P` and the target device has an Arm® Cortex®-M7 CPU 1.*

Note: *`MCU_POWERMODE_RETAINED` can be effective only when Main Core 1 is in CPU DeepSleep mode.*

- `McuSleepOnExitIsrEnable` enables or disables the CPU entering Sleep state on exiting from an ISR.
- `McuWakeupByPendingInterruptEnable` enables or disables the CPU waking up by an interrupt transition from an inactive state to pending state.
- `McuEnableCacheFlushBeforeModeChange` enables or disables flushing cache before changing mode.
- `McuRamWriteBufferTimeout` specifies the timeout count value used when checking whether the RAM write buffer status is empty.
 - 1 - 4294967295: Timeout count value is used when checking that the RAM write buffer is empty.
- `McuFreezeIoRelease` enables or disables releasing the I/O freeze.

Note: *If I/O freeze is enabled when entering Hibernate mode, then after wakeup, I/O freeze should be released by applying the mode configuration with this parameter set to `TRUE`.*

- `McuUpdateSystemResource` specifies whether to update the system resources or not.

Note: *The following parameters are related to system resources controlled by this parameter. If this parameter is `FALSE`, the following parameters are not applied. The system resources should be updated from only one (master) CPU core.*

- `McuReferenceClockSetting`
- `McuLinearCoreRegulatorDisable`
- `McuLinearCoreRegulatorEnableTimeout`
- `McuDeepSleepRegulatorDisable`
- `McuVoltageReferenceBufferDisable`
- `McuVoltageReferenceBufferReadyTimeout`
- `McuReferenceCurrentGeneratorDisable`
- `McuReferenceCurrentGeneratorEnableTimeout`
- `McuBandgapReferencePowerMode`

- `McuBypassPllLevelShifter`
- `McuHvLvdSettings`
- `McuRegHcSettings`
- `McuReferenceClockSetting` selects the clock setting configuration from `McuClockSettingConfig`, which is applied to its mode configuration.
- `McuMainCore0PowerUpDelay` specifies the delay after power up of Main Core 0 power domain in clock cycles.
 - 0 - 1023: Delay count in cycles.
- `McuMainCore1PowerUpDelay` specifies the delay after power up of Main Core 1 power domain in clock cycles.
 - 0 - 1023: Delay count in cycles.
- `McuRam0Macro<n>PowerMode` (<n> = 0 ... 15) specifies the RAM0 Macro <n> power mode.
 - `MCU_POWERMODE_OFF`: Switch OFF.
 - `MCU_POWERMODE_RETAINED`: Put in retained mode.
 - `MCU_POWERMODE_ENABLED`: Switch ON.

Note: *Selectable RAM0 macros depend on the subderivative.*

- Some SRAM areas may be used by the SROM API. So, the power of those SRAM areas should not be disabled when the SROM API is used. If some of the SRAM0 areas are used by the SROM API, `McuRam0Macro<n>PowerMode` corresponding to those areas should not be configured to `MCU_POWERMODE_OFF`. `McuRam1PowerMode` specifies the RAM1 power mode.
 - `MCU_POWERMODE_OFF`: OFF mode
 - `MCU_POWERMODE_RETAINED`: Retained mode
 - `MCU_POWERMODE_ENABLED`: ON mode

Note: *Some SRAM areas may be used by the SROM API. So, the power of those SRAM areas should not be disabled when the SROM API is used. If the SRAM1 areas are used by the SROM API, `McuRam1PowerMode` should not be configured to `MCU_POWERMODE_OFF`.*

- `McuRam2PowerMode` specifies the RAM2 power mode.
 - `MCU_POWERMODE_OFF`: OFF mode
 - `MCU_POWERMODE_RETAINED`: Retained mode
 - `MCU_POWERMODE_ENABLED`: ON mode

Note: *Some SRAM areas may be used by the SROM API. So, the power of those SRAM areas should not be disabled when the SROM API is used. If the SRAM2 areas are used by the SROM API, `McuRam2PowerMode` should not be configured to `MCU_POWERMODE_OFF`.*

- `McuRamPowerUpDelay` specifies the delay after power up of all RAM power domain in cycles.
 - 0 - 1023: Delay count in cycles.
- `McuLowPowerReadyTimeout` specifies the timeout count value used when checking that low-power functions is ready.
 - 1 - 4294967295: Timeout count value used when checking that the low-power function is ready.
- `McuLinearCoreRegulatorDisable` enables or disables the linear core regulator.

Note: *If this parameter is TRUE, the linear core regulator will be disabled.*

Note: *This parameter must be set to FALSE.*

- `McuLinearCoreRegulatorEnableTimeout` specifies the timeout count value used when checking that the linear core regulator is ready.
 - 1 - 4294967295: Timeout count value used when checking that the linear core regulator is ready.
- `McuDeepSleepRegulatorDisable` disables or enables the DeepSleep regulator.

Note: *If this parameter is TRUE, the DeepSleep regulator will be disabled. Once the DeepSleep regulator is disabled, it will not be enabled again later.*

- `McuVoltageReferenceBufferDisable` enables or disables the voltage reference buffer.

Note: *If this parameter is TRUE, the voltage reference buffer will be disabled.*

- `McuVoltageReferenceBufferReadyTimeout` specifies the timeout count value used when checking that the voltage reference buffer is ready.
 - 1 - 4294967295: Timeout count value used when checking that the voltage reference buffer is ready.
- `McuReferenceCurrentGeneratorDisable` disables or enables the reference current generator.

Note: *If this parameter is TRUE, the reference current generator will be disabled.*

Note: *This parameter must be set to FALSE.*

- `McuReferenceCurrentGeneratorEnableTimeout` specifies the timeout count value used when checking that the reference current generator is ready.
 - 1 - 4294967295: Timeout count value used when checking that the reference current generator is ready.
- `McuBandgapReferencePowerMode` specifies the power mode of the bandgap reference circuits.
 - `MCU_POWERMODE_NORMAL`: Normal mode
 - `MCU_POWERMODE_LOWPOWER`: Low-power mode
- `McuBypassPllLevelShifter` specifies whether bypass level shifter inside the PLL or not.

MCU mode settings configuration holds the following containers.

- `McuHibernateSettings` (see [MCU hibernate mode settings](#))
- `McuSupplySupervisionSettings` (see [MCU supply supervision settings](#))
- `McuHvLvdSettings` (see [MCU HVLVD settings](#))
- `McuRegHcSettings` (see [MCU REGHC settings](#))
- `McuPmicSettings` (see [MCU PMIC settings](#))
- `McuDmaSettings` (see [MCU DMA settings](#))

4.6.1 MCU hibernate mode settings

`McuHibernateSettings` container has the following parameters to configure the Hibernate mode:

- `McuHibernateFreezeIoEnable` enables or disables the I/O freeze when entering the Hibernate mode.
- `McuHibernateWakeupByBackupAlarmEnable` enables or disables the wake up from Hibernate mode by a RTC interrupt.
- `McuHibernateWakeupByWatchdogEnable` enables or disables the wake up from Hibernate mode by a WDT.
- `McuHibernateWakeupByBackupCsvEnable` enables or disables the wake up from Hibernate mode by a backup clock supervisor.
- `McuHibernateWakeupSenseMode` enables or disables the wake up from Hibernate mode by the pending interrupt.
- `McuHibernateWakeupByWakeupPin<n>Enable` (<n> = 0 ... 23) enables or disables the wake up from Hibernate mode by the wakeup pin input. The wakeup will occur when its input matches `McuHibernateWakeupPin<n>Polarity`.
- `McuHibernateWakeupPin<n>Polarity` (<n> = 0 ... 23) specifies the active polarity of the corresponding wakeup pin.
 - `MCU_PIN_POLARITY_LOW`: Pin input of 0 will trigger the wakeup from Hibernate mode.
 - `MCU_PIN_POLARITY_HIGH`: Pin input of 1 will trigger the wakeup from Hibernate mode.

Note: *If this container is available only if `McuCpuPowerMode` is `MCU_CPUMODE_HIBERNATE`.*

4.6.2 MCU HVLVD settings

The `McuHvLvdSettings` container has the following parameters to configure the HVLVD:

- `McuHvLvdType` specifies the HVLVD type.
 - `MCU_HVLVD_HVLVD1`: HVLVD1
 - `MCU_HVLVD_HVLVD2`: HVLVD2

Note: *In the same `McuModeSettingConf` container, `McuHvLvdType` must be unique.*

- `McuHvLvdEnable` enables or disables the HVLVD.
- `McuHvLvdOnDeepSleepEnable` keeps the HVLVD specified by `McuHvLvdType` enabled during DeepSleep mode.

Note: *If this parameter is `TRUE`, `McuHvLvdEnable` must be `TRUE`.*

- `McuHvLvdStopForUpdate` stops HVLVD specified by `McuHvLvdType` once before setting the HDLVD by configuration.

Note: *If `McuHvLvdStopForUpdate` is `FALSE`, setting the HVLVD will be skipped when it is running.*

- `McuHvLvdThreshold` specifies the threshold value of the HVLVD specified by `McuHvLvdType`.
 - `MCU_HVLVD_THRESHOLD_2_8V_TO_2_825V`: 2.8[V] to 2.825[V]
 - `MCU_HVLVD_THRESHOLD_2_9V_TO_2_925V`: 2.9[V] to 2.925[V]
- `McuHvLvdAction` specifies the action executed when the error is detected by the HVLVD specified by `McuHvLvdType`.
 - `MCU_LVD_ACTION_FAULT`: Fault report.

- `MCU_LVD_ACTION_INTERRUPT`: Interrupt.

Note: When `MCU_LVD_ACTION_FAULT` is configured, you should handle the fault report of HVLVD.

- `McuHvLvdInterruptEnable` enables or disables the interrupt of the HVLVD specified by `McuHvLvdType`.
- `McuHvLvdTriggerEdge` specifies the edge which triggers an action when the threshold is crossed.
 - `MCU_HVLVD_EDGE_RISING`: Rising edge.
 - `MCU_HVLVD_EDGE_FALLING`: Falling edge.
 - `MCU_HVLVD_EDGE_BOTH`: Both edges.

4.6.3 MCU supply supervision settings

The `McuSupplySupervisionSettings` container has the following parameters to configure the supply supervision:

- `McuVdddBodEnable` enables or disables the BOD on VDDD.

Note: The BOD on VDDD cannot be disabled, so this parameter is always `TRUE`.

- `McuVdddBodThreshold` specifies the threshold value of the BOD on VDDD.
 - `MCU_BOD_THRESHOLD_2_7V`: 2.7[V]
 - `MCU_BOD_THRESHOLD_3_0V`: 3.0[V]
- `McuVddaBodEnable` enables or disables the BOD on VDDA.
- `McuVddaBodThreshold` specifies the threshold value of the BOD on VDDA.
 - `MCU_BOD_THRESHOLD_2_7V`: 2.7[V]
 - `MCU_BOD_THRESHOLD_3_0V`: 3.0[V]
- `McuVddaBodAction` specifies the BOD on VDDA action.
 - `MCU_BOD_ACTION_NONE`: No action.
 - `MCU_BOD_ACTION_RESET`: Reset.
 - `MCU_BOD_ACTION_FAULT`: Fault report.

Note: When `MCU_BOD_ACTION_FAULT` is configured, you should handle the fault report of BOD.

- `McuVccdBodEnable` enables or disables the BOD on VCCD.

Note: The BOD on VCCD cannot be disabled, so this parameter is always `TRUE`.

- `McuVdddOvdEnable` enables or disables the OVD on VDDD.

Note: The OVD on VDDD cannot be disabled, so this parameter is always `TRUE`.

- `McuVdddOvdThreshold` specifies the threshold value of the OVD on VDDD.
 - `MCU_OVD_THRESHOLD_5_0V`: 5.0[V]
 - `MCU_OVD_THRESHOLD_5_5V`: 5.5[V]
- `McuVddaOvdEnable` enables or disables the OVD on VDDA.
- `McuVddaOvdThreshold` specifies the threshold value of the OVD on VDDA.
 - `MCU_OVD_THRESHOLD_5_0V`: 5.0[V]
 - `MCU_OVD_THRESHOLD_5_5V`: 5.5[V]

- `McuVddaOvdAction` specifies the OVD on VDDA action.
 - `MCU_OVD_ACTION_NONE`: no action
 - `MCU_OVD_ACTION_RESET`: Cause a reset
 - `MCU_OVD_ACTION_FAULT`: Cause a fault report

Note: When `MCU_OVD_ACTION_FAULT` is configured, you should handle the fault report of OVD.

- `McuVccdOvdEnable` enables or disables the OVD on VCCD.

Note: The OVD on VCCD cannot be disabled, so this parameter is always `TRUE`.

4.6.4 MCU REGHC settings

The `McuRegHcSettings` container has the following parameters to configure the REGHC:

Note: The usage of this functionality in MCAL is not recommended. For the implementation of REGHC, see AN226698 - External Power Supply Design Guide for TRAVEO™ II Family.

Note: The REGHC settings except for enabling are one-time writable. So MCU driver only enables or disables the REGHC and PMIC VADJ for REGHC. If you use REGHC, setup those registers in startup or user code.

- `McuRegHcEnable` enables or disables the REGHC.
- `McuRegHcOnDeepSleepEnable` keeps the REGHC enabled during DeepSleep mode.
- `McuRegHcPmicVadjDisable` disables or enables the PMIC VADJ for REGHC.
- `McuRegHcStabilizationTimeout` specifies the timeout count value used when checking that the REGHC is stabilized.
 - 1 - 4294967295: Timeout count value used when checking that the REGHC is stabilized.

4.6.5 MCU PMIC settings

The `McuPmicSettings` container has the following parameters to configure the PMIC:

Note: The usage of this functionality in MCAL is not recommended. For the implementation of PMIC, see AN226698 - External power supply design guide for TRAVEO™ II family.

Note: The PMIC settings except for enabling are one-time writable. So MCU driver only enables or disables the PMIC and PMIC VADJ. If you use PMIC, setup those registers in startup or user code.

- `McuPmicEnable` enables or disables the PMIC.
- `McuPmicOnDeepSleepEnable` keeps the PMIC enabled during DeepSleep mode.
- `McuPmicVadjDisable` disables or enables the PMIC VADJ.
- `McuPmicStabilizationTimeout` specifies the timeout count value used when checking that the PMIC is stabilized.
 - 1 - 4294967295: Timeout count value used when checking that the PMIC is stabilized.

4.6.6 MCU DMA settings

The `McuDmaSettings` container has the following parameters to configure the DMA:

- `McuDmaEnable` enables or disables the DMA.
- `McuDataWire0Enable` enables or disables the Data Wire 0.
- `McuDataWire1Enable` enables or disables the Data Wire 1.

4.7 MCU RAM section configuration

The `McuRamSectorSettingConf` container has the following parameters to configure the RAM section settings:

- `McuRamSectionBaseAddress` specifies the address where the RAM section to initialize starts.
- `McuRamSectionSize` specifies the size of this RAM section.
- `McuRamDefaultValue` specifies the initialization value for the RAM section. It is 8 bits long.

4.8 MCU published information

The `McuPublishedInformation` container has different types of reset reasons that can be retrieved from the `Mcu_GetResetReason()` API. This container is not editable. The `McuResetReason` values are assigned to the following symbolic name.

The symbolic name derived from the `McuResetReasonConf` container short name is prefixed with `"McuConf_McuResetReasonConf_"`.

Example:

`McuConf_McuResetReasonConf_MCU_RESET_UNDEFINED.`

Table 1 List of reset reasons

Container	McuResetReason value
MCU_RESET_UNDEFINED	0
MCU_POWER_ON_RESET	1
MCU_WATCHDOG_RESET	2
MCU_ACT_FAULT_RESET	3
MCU_DPSLP_FAULT_RESET	4
MCU_TEST_DEBUG_RESET	5
MCU_SW_RESET	6
MCU_MCWDT0_RESET	7
MCU_MCWDT1_RESET	8
MCU_MCWDT2_RESET	9
MCU_MCWDT3_RESET	10
MCU_XRES_RESET	11
MCU_BOD_VDDD_RESET	12
MCU_BOD_VDDA_RESET	13
MCU_BOD_VCCD_RESET	14
MCU_OVD_VDDD_RESET	15
MCU_OVD_VDDA_RESET	16
MCU_OVD_VCCD_RESET	17
MCU_OCD_ACTIVE_REGULATOR_RESET	18
MCU_OCD_DEEPSLEEP_REGULATOR_RESET	19
MCU_STRUCTURAL_XRES_RESET	20
MCU_CSV_HF_RESET	21
MCU_CSV_REF_RESET	22
MCU_WAKEUP_RESET	23
MCU_REGHC_OCD_RESET	24
MCU_REGHC_PMIC_RESET	25
MCU_PXRES_RESET	26

5 Functional description

5.1 Inclusion

The file *Mcu.h* includes all necessary external identifiers. Therefore, the application only needs to include *Mcu.h* to make all API functions and data types available.

The clock setting is done by the `Mcu_InitClock` API and the low-power mode setting is done by the `Mcu_SetMode` API. Both CPU cores need to be initialized, so the application in each code needs to include *Mcu.h*.

5.2 Initialization

The MCU driver provides an initialization function for initializing the μ C's CPU core. As it is possible to set more than one configuration, the `Mcu_Init()` function can be called with different configuration sets.

```
Mcu_Init(&Mcu_Config[0]);
```

Example:

A clock setup can be accomplished by calling the following function:

```
Mcu_InitClock(McuConf_McuClockSettingConfig_MY_CLOCK);
```

Note: See [Appendix B – Access register table](#) for the registers that will be initialized by the MCU module. If you need to initialize the registers that other than listed in [Appendix B – Access register table](#), they should be initialized by each MCAL module or startup.

Example:

This initializes the clock with the selected configuration. On this architecture, a switch to the PLL is already performed during the initialization of the clock when a configuration with PLL is given.

```
Mcu_DistributePllClock();
```

Note: Clock settings that are not set by the Mcu module configuration are not set by the MCU module API. If it is necessary to disable the specific clock, you should disable that clock in the configuration. If you need to set the clock trimming values, control the values in startup or user code. If you need to disable the slave of peripheral group as a system, control the slave of the peripheral group in startup.

This function distributes the FLL, PLLs, SSCG PLLs, or all.

Note: Only the FLL, PLLs, and/or SSCG PLLs which are set by preceding `Mcu_InitClock()` or `Mcu_SetMode()` are processed in `Mcu_GetPllStatus()` and `Mcu_DistributePllClock()`.

5.3 MCU mode

The MCU driver provides a function that sets the microcontroller into a low-power mode:

```
Mcu_SetMode (McuConf_McuModeSettingConf_MY_MODE);
```

Example:

This function sets the microcontroller with the specified mode.

Note: *If you need to disable the Hibernate mode permanently in the system, control the Hibernate mode in startup. Set `HIBERNATE_DISABLE` bit of the `PWR_HIBERNATE` register to disable the Hibernate mode.*

When entering Hibernate mode, you should execute the WFI instruction on all cores except for the core that `Mcu_SetMode()` is called.

The REGHC settings except for enabling are one-time writable. So MCU driver only enables or disables the REGHC. If you use REGHC, setup those registers in startup or user code.

If you use the DW or the DMA for other modules, you can enable them by using MCU driver. MCU driver does not control the each DW channel and DMA channel. They would be enabled by other modules that use them.

When FLL, PLLs, SSCG PLLs or all are enabled by `Mcu_SetMode()`, it may be necessary to call `Mcu_GetPllStatus()` and `Mcu_DistributePllClock()` after calling `Mcu_SetMode()`. For example, the case in which stabilization wait of FLL, PLLs, SSCG PLLs or all are disabled.

Basically, only `Mcu_Init()` and `Mcu_SetMode()` can be called from slave CPU core.

5.4 API parameter checking

The driver's services perform error checks.

When an error occurs, the error hook routine (configured via `McuErrorCalloutFunction`) is called and the error code, service ID, module ID, and instance ID are passed as parameters.

If development error detection is enabled, all errors are also reported to the DET, a central error hook function within the AUTOSAR environment. The checking itself cannot be deactivated for safety reasons.

The following development error checks are performed by the services of the MCU driver:

- The API function `Mcu_Init()` checks if the given parameter is within the valid range to select a configuration. If the parameter is invalid, the `MCU_E_INIT_FAILED` error is reported.
- All API functions except `Mcu_Init()` and `Mcu_GetVersionInfo()` report the error `MCU_E_UNINIT` if the MCU driver has not been initialized properly yet.
- The API functions `Mcu_SetMode()` and `Mcu_CheckModeStatus()` check if the given parameter is within the valid range to select a configuration. If the parameter is invalid, the `MCU_E_PARAM_MODE` error is reported.
- The API functions `Mcu_InitClock()` and `Mcu_CheckClockStatus()` check if the given parameter is within the valid range to select a configuration. If the parameter is invalid, the `MCU_E_PARAM_CLOCK` error is reported.
- The API function `Mcu_InitRamSection()` checks if the given parameter is within the valid range to select a configuration. If the parameter is invalid, the `MCU_E_PARAM_RAMSECTION` error is reported.
- The API function `Mcu_GetVersionInfo()` checks if the function is called with a NULL pointer. If the parameter is a NULL pointer, the error code `MCU_E_PARAM_POINTER` is reported.
- The API function `Mcu_CheckModeStatus()` checks if the given parameter is a NULL pointer. If the parameter is a NULL pointer, the `MCU_E_PARAM_POINTER` error is reported.

- The API function `Mcu_DistributePllClock()` reports the error `MCU_E_PLL_NOT_LOCKED`, if the status of the PLL is not locked.
- The API functions `Mcu_PerformReset()` reports the error `MCU_E_RESET_NOT_PERFORMED`, if `McuResetSelect` is not configured.
- The API function `Mcu_SetMode()` reports the error `MCU_E_PARAM_MODE` if the clock setting fails.
- The API function `Mcu_SetMode()` reports the error `MCU_E_SYSTEM_RESOURCE_UPDATE_NOT_COMPLETED`, if the update of the system common resources is not completed.

5.5 Production error detection

If clock source failure occurs, `MCU_E_CLOCK_FAILURE` is reported to the DEM.

If reset failure occurs, `MCU_E_RESET_FAILURE` is reported to the DEM.

When an error occurs, the error hook routine (configured via `McuErrorCalloutFunction`) is also called and the error code (`MCU_E_CLOCK_FAILURE_FOR_CALLOUT` or `MCU_E_RESET_FAILURE_FOR_CALLOUT`), service ID, module ID, and instance ID are passed as parameters.

5.6 Reentrancy

The following functions are reentrant to each other and itself. All other API functions of the MCU driver are not reentrants:

- `Mcu_GetResetRawValue()`
- `Mcu_GetResetReason()`
- `Mcu_GetPllStatus()`
- `Mcu_GetVersionInfo()`
- `Mcu_CheckClockStatus()`
- `Mcu_CheckModeStatus()`

5.7 Debugging support

The MCU driver does not support debugging.

5.8 APIs requiring privileged execution

Following APIs require privileged execution because they access the registers which require privileged access:

- `Mcu_SetMode()`
- `Mcu_PerformReset()`

6 Hardware resources

6.1 Timer

The MCU driver does not use hardware timers.

6.2 Interrupts

The MCU driver uses the nonmaskable interrupts for low-voltage detection. The ISR must be declared in the AUTOSAR OS as Category 1 Interrupt or Category 2 Interrupt.

Note: Vector numbers depend on the subderivative.

To define the ISR, the IRQ name of the nonmaskable interrupt for low-voltage detection must be `Mcu_Lvd_Isr_Cat1()` for Category 1 ISR or `Mcu_Lvd_Isr_Cat2()` for Category 2 ISR.

*Note: `Mcu_SyscNmiCsv_Cat2()` and `Mcu_SyscNmiLvd_Cat2()` must be called from the (OS) interrupt service routine.
For Category 1 usage, the address of `Mcu_SyscNmiCsv_Cat1()` and `Mcu_SyscNmiLvd_Cat1()` must be the entry in the (OS) NMI interrupt vector table.*

Example: Category 1 ISR for LVD located in file `generate/src/Mcu_Irq.c`:

```
ISR_NATIVE(Mcu_Lvd_Isr_Cat1)
{
    ...
}
```

Example: Category 2 ISR for LVD located in file `generate/src/Mcu_Irq.c`:

```
ISR(Mcu_Lvd_Isr_Cat2)
{
    ...
}
```

Note: On the Arm® Cortex®-M4 CPU, priority inversion of interrupts may occur under specific timing conditions in the integrated system with TRAVERO™ II MCAL. For more details, see the following errata notice.

Arm® Cortex®-M4 Software Developers Errata Notice - 838869:

“Store immediate overlapping exception return operation might vector to incorrect interrupt”

If the user application cannot tolerate the priority inversion, a DSB instruction should be added at the end of the interrupt function to avoid the priority inversion.

TRAVERO™ II MCAL interrupts are handled by an ISR wrapper (handler) in the integrated system. Thus, if necessary, the DSB instruction should be added just before the end of the handler by the integrator.

6.3 Fault report structure

The MCU driver does not use the fault report structure.

But, the hardware configured by the MCU driver can use the fault report structure to report errors. For example, when `McuCsvAction` is configured to `MCU_CSV_ACTION_FAULT` and the clock supervisor detects the error, the fault report structure reports the error.

To handle this, you should implement the handler for the fault report structure.

For details on the fault report structure and its assignment, see the architecture TRM and datasheet.

7 Appendix A – API reference

7.1 Data types

7.1.1 Mcu_ConfigType

Type

```
typedef struct
```

Description

Mcu_ConfigType defines a structure which holds the MCU driver configuration set.

7.1.2 Mcu_PllStatusType

Type

```
typedef enum
{
    MCU_PLL_STATUS_UNDEFINED,
    MCU_PLL_UNLOCKED,
    MCU_PLL_LOCKED
} Mcu_PllStatusType;
```

Description

Mcu_PllStatusType defines the values that describe the status of the PLL.

7.1.3 Mcu_ClockType

Type

```
uint8
```

Description

Mcu_ClockType defines the range of different clock settings provided in the configuration structure. It is used as an index for selection clock configurations for `Mcu_InitClock()`.

7.1.4 Mcu_ResetType

Type

```
typedef enum (see Table 1 for contents)
```

Description

Mcu_ResetType defines the subset of reset types.

7.1.5 Mcu_RawResetType

Type

uint32

Description

`Mcu_RawResetType` defines the reset reason in raw register format read from a reset status register. The values of `Mcu_RawResetType` depend on hardware. For details of these values, see the information on reset result register in the hardware manual.

7.1.6 Mcu_ModeType

Type

uint8

Description

`Mcu_ModeType` defines the range of different MCU modes provided in the configuration structure.

7.1.7 Mcu_RamSectionType

Type

uint16

Description

`Mcu_RamSectionType` defines the range of different RAM sections provided in the configuration structure.

7.1.8 Mcu_RamStateType

Type

```
typedef enum
{
    MCU_RAMSTATE_INVALID,
    MCU_RAMSTATE_VALID
} Mcu_RamStateType;
```

Description

`Mcu_RamStateType` defines the values that describe the status of the RAM.

7.1.9 Mcu_StatusType

Type

```
typedef struct
{
    Mcu_CpuStatusType    Cm0Status;
    Mcu_CpuStatusType    MainCoreStatus[2];
    Mcu_SysStatusType    SysStatus;
}Mcu_StatusType;
```

Description

`Mcu_StatusType` defines the result of status check.

Cm0Status: CM0P CPU status.

MainCoreStatus[2]: Main core CPU status. If there is only one core in the system, only the first of the array is used.

SysStatus: System status.

7.1.10 Mcu_CpuStatusType

Type

uint8

Description

Mcu_CpuStatusType defines the CPU status.

7.1.11 Mcu_SysStatusType

Type

uint8

Description

Mcu_SysStatusType defines the system status.

7.2 Constants

7.2.1 Error codes

The service might return the error codes, shown in [Table 2](#), if development error detection is enabled:

Table 2 Error codes

Name	Value	Description
MCU_E_PARAM_CLOCK	0x0B	ClockSetting is not a valid parameter.
MCU_E_PARAM_MODE	0x0C	McuMode is not a valid parameter.
MCU_E_PARAM_RAMSECTION	0x0D	RamSection is not a valid parameter.
MCU_E_PLL_NOT_LOCKED	0x0E	PLL not locked yet.
MCU_E_UNINIT	0x0F	MCU has not been initialized yet.
MCU_E_PARAM_POINTER	0x10	versioninfo is a NULL pointer.
MCU_E_INIT_FAILED	0x11	ConfigPtr is a wrong parameter.
MCU_E_CLOCK_FAILURE_FOR_CALLOUT	0x40	Clock source failure is occurred. This error id is used to call the error callout handler.
MCU_E_RESET_FAILURE_FOR_CALLOUT	0x41	Reset failure is occurred. This error id is used to call the error callout handler
MCU_E_RESET_NOT_PERFORMED	0x60	Mcu_PerformReset doesn't perform reset.
MCU_E_SYSTEM_RESOURCE_UPDATE_NOT_COMPLETED	0x80	System resource update does not complete.

7.2.2 Version information

Table 3 lists the version information published in the driver's header file.

Table 3 Version information

Name	Value	Description
MCU_SW_MAJOR_VERSION	Refer to release notes	Major version number
MCU_SW_MINOR_VERSION	Refer to release notes	Minor version number
MCU_SW_PATCH_VERSION	Refer to release notes	Patch version number

7.2.3 Module information

Table 4 Module information

Name	Value	Description
MCU_MODULE_ID	101	Module ID
MCU_VENDOR_ID	66	Vendor ID

7.2.4 API service IDs

The API service IDs, listed in **Table 5**, are published in the driver's header file.

Table 5 API service IDs

Name	Value	Description
MCU_API_SERVICE_INIT	0x0	Service ID of <code>Mcu_Init</code>
MCU_API_SERVICE_INIT_RAM_SECTION	0x1	Service ID of <code>Mcu_InitRamSection</code>
MCU_API_SERVICE_INIT_CLOCK	0x2	Service ID of <code>Mcu_InitClock</code>
MCU_API_SERVICE_DISTRIBUTE_PLL_CLOCK	0x3	Service ID of <code>Mcu_DistributePllClock</code>
MCU_API_SERVICE_GET_PLL_STATUS	0x4	Service ID of <code>Mcu_GetPllStatus</code>
MCU_API_SERVICE_GET_RESET_REASON	0x5	Service ID of <code>Mcu_GetResetReason</code>
MCU_API_SERVICE_GET_RESET_RAW_VALUE	0x6	Service ID of <code>Mcu_GetResetRawValue</code>
MCU_API_SERVICE_PERFORM_RESET	0x7	Service ID of <code>Mcu_PerformReset</code>
MCU_API_SERVICE_SET_MODE	0x8	Service ID of <code>Mcu_SetMode</code>
MCU_API_SERVICE_GET_VERSION_INFO	0x9	Service ID of <code>Mcu_GetVersionInfo</code>
MCU_API_SERVICE_CHECK_CLOCK_STATUS	0x20	Service ID of <code>Mcu_CheckClockStatus</code>
MCU_API_SERVICE_CHECK_MODE_STATUS	0x21	Service ID of <code>Mcu_CheckModeStatus</code>

7.3 Functions

7.3.1 Mcu_Init

Syntax

```
void Mcu_Init(  
    const Mcu_ConfigType* ConfigPtr  
)
```

Service ID

0x0

Parameters (in)

ConfigPtr

Parameters (out)

None

Return value

None

DET errors

MCU_E_INIT_FAILED - Invalid parameter.

DEM errors

MCU_E_CLOCK_FAILURE - Clock source failure is occurred.

Description

This function initializes the MCU driver and shows the configuration settings for power down, clock, and RAM sections within the MCU driver.

7.3.2 Mcu_InitRamSection

Syntax

```
Std_ReturnType Mcu_InitRamSection(  
    Mcu_RamSectionType RamSection  
)
```

Service ID

0x1

Parameters (in)

RamSection - Selects the RAM memory section provided in the configuration set.

Parameters (out)

None

Return value

E_OK or E_NOT_OK

DET errors

- `MCU_E_PARAM_RAMSECTION` - Invalid parameter.
- `MCU_E_UNINIT` - The module is uninitialized.

DEM errors

None

Description

This function initializes the RAM section wise.

7.3.3 Mcu_InitClock

Syntax

```
Std_ReturnType Mcu_InitClock(  
    Mcu_ClockType ClockSetting  
)
```

Service ID

0x2

Parameters (in)

`ClockSetting` - Clock setting.

Parameters (out)

None

Return value

`E_OK` or `E_NOT_OK`

DET errors

- `MCU_E_PARAM_CLOCK` - Invalid parameter.
- `MCU_E_UNINIT` - The module is uninitialized.

DEM errors

`MCU_E_CLOCK_FAILURE` - Clock source failure.

Description

This function initializes the PLL and other MCU-specific clock options.

7.3.4 Mcu_DistributePllClock

Syntax

```
Std_ReturnType Mcu_DistributePllClock(  
    void  
)
```

Service ID

0x3

Parameters (in)

None

Parameters (out)

None

Return value

E_OK or E_NOT_OK

DET errors

- MCU_E_PLL_NOT_LOCKED - The status of the PLL is not locked.
- MCU_E_UNINIT - The module is uninitialized.

DEM errors

None

Description

This function activates the FLL, PLLs, SSCG PLLs or all to the MCU clock distribution. This function is executed if the MCU needs a separate request to activate the FLL, PLLs, or both after the FLL, PLLs, SSCG PLLs or all are locked.

7.3.5 Mcu_GetPllStatus

Syntax

```
Mcu_PllStatusType Mcu_GetPllStatus(  
    void  
)
```

Service ID

0x4

Parameters (in)

None

Parameters (out)

None

Return value

The lock status of the PLL clock.

DET errors

MCU_E_UNINIT - The module is uninitialized.

DEM errors

None

Description

This function provides the lock status of the PLLs, SSCG PLLs or the FLL.

7.3.6 Mcu_GetResetReason

Syntax

```
Mcu_ResetType Mcu_GetResetReason(  
    void  
)
```

Service ID

0x5

Parameters (in)

None

Parameters (out)

None

Return value

Reset reason

DET errors

MCU_E_UNINIT - The module is uninitialized.

DEM errors

None

Description

This function returns the reset reason, if supported by hardware. A call to the API service returns exactly one reset reason. If no more reset reasons are available, the reset cause `MCU_RESET_UNDEFINED` is returned.

7.3.7 Mcu_GetResetRawValue

Syntax

```
Mcu_RawResetType Mcu_GetResetRawValue(  
    void  
)
```

Service ID

0x6

Parameters (in)

None

Parameters (out)

None

Return value

Raw reset type

DET errors

MCU_E_UNINIT - The module is uninitialized.

DEM errors

None

Description

This function reads the reset type from the hardware register, if supported.

7.3.8 Mcu_PerformReset

Syntax

```
void Mcu_PerformReset(  
    void  
)
```

Service ID

0x7

Parameters (in)

None

Parameters (out)

None

Return value

None

DET errors

- MCU_E_UNINIT - The module is uninitialized.
- MCU_E_RESET_NOT_PERFORMED - McuResetSelect is not configured.

DEM errors

MCU_E_RESET_FAILURE - Reset failure is occurred.

Description

This function performs a microcontroller reset, whereby the hardware feature of the microcontroller is used.

7.3.9 Mcu_SetMode

Syntax

```
void Mcu_SetMode(  
    Mcu_ModeType McuMode  
)
```

Service ID

0x8

Parameters (in)

`McuMode` - Selects the mode configured in the configuration set.

Parameters (out)

None

Return value

None

DET errors

- `MCU_E_PARAM_MODE` - Invalid parameter.
- `MCU_E_UNINIT` - The module is uninitialized.
- `MCU_E_SYSTEM_RESOURCE_UPDATE_NOT_COMPLETED` - System resource update error.

DEM errors

`MCU_E_CLOCK_FAILURE` - Clock source failure has occurred.

Description

This function sets the microcontroller into a low-power mode.

7.3.10 `Mcu_GetVersionInfo`

Syntax

```
void Mcu_GetVersionInfo(  
    Std_VersionInfoType* versioninfo  
)
```

Service ID

0x9

Parameters (in)

None

Parameters (out)

`versioninfo` - Version information of the MCU driver.

Return value

None

DET errors

`MCU_E_PARAM_POINTER` - Parameter `versioninfo` is a NULL pointer.

DEM errors

None

Description

This function returns the version of this module.

7.3.11 Mcu_CheckClockStatus

Syntax

```
Std_ReturnType Mcu_CheckClockStatus(  
    Mcu_ClockType ClockSettingId,  
)
```

Service ID

0x20

Parameters (in)

ClockSettingId - Clock setting ID for checking.

Parameters (out)

None

Return value

E_OK or E_NOT_OK

DET errors

- MCU_E_UNINIT - The module is uninitialized.
- MCU_E_PARAM_CLOCK - Invalid parameter.

DEM errors

None

Description

This service checks whether the register has value corresponding to the clock configuration.

7.3.12 Mcu_CheckModeStatus

Syntax

```
Std_ReturnType Mcu_CheckModeStatus(  
    Mcu_ModeType ModeSettingId,  
    Mcu_StatusType* StatusPtr  
)
```

Service ID

0x21

Parameters (in)

ModeSettingId - Mode setting ID for checking.

Parameters (out)

StatusPtr - Result of status check.

Return value

E_OK or E_NOT_OK

DET errors

- `MCU_E_UNINIT` - The module is uninitialized.
- `MCU_E_PARAM_MODE` - Invalid parameter.
- `MCU_E_PARAM_POINTER` - Parameter `StatusPtr` is a NULL pointer.

DEM errors

None

Description

This service checks whether the register has a value corresponding to the mode configuration.

7.4 Required callback functions

7.4.1 DET

If development error detection is enabled, the MCU driver uses the following callback function provided by DET. If you do not use DET, you must implement this function within your application.

7.4.1.1 Det_ReportError

Syntax

```
Std_ReturnType Det_ReportError  
(  
    uint16 ModuleId,  
    uint8 InstanceId,  
    uint8 ApiId,  
    uint8 ErrorId  
)
```

Reentrancy

Reentrant

Parameters (in)

- `ModuleId` - Module ID of calling module.
- `InstanceId` - Instance ID of calling module.
- `ApiId` - ID of the API service that calls this function.
- `ErrorId` - ID of the detected development error.

Return value

Returns always `E_OK` (is required for services).

Description

Service for reporting development errors.

7.4.2 DEM

If DEM notifications are enabled, the MCU driver uses the following callback function provided by DEM. If you do not use DEM, you must implement this function within your application.

7.4.2.1 Dem_ReportErrorStatus

Syntax

```
void Dem_ReportErrorStatus
(
    Dem_EventIdType EventId,
    Dem_EventStatusType EventStatus
)
```

Reentrancy

Reentrant

Parameters (in)

- `EventId` - Identification of an event by assigned event ID.
- `EventStatus` - Monitor test result of given event.

Return value

None

Description

Service for reporting diagnostic events.

7.4.3 Callout functions

7.4.3.1 Error callout API

The AUTOSAR MCU module requires an error callout handler. Each error is reported to this handler; error checking cannot be switched OFF. The name of the function to be called can be configured with the `McuErrorCalloutFunction` parameter.

Syntax

```
void Error_Handler_Name
(
    uint16 ModuleId,
    uint8 InstanceId,
    uint8 ApiId,
    uint8 ErrorId
)
```

Reentrancy

Reentrant

Parameters (in)

- `ModuleId` - Module ID of calling module.
- `InstanceId` - Instance ID of calling module.
- `ApiId` - ID of the API service that calls this function.

- `ErrorId` - ID of the detected error.

Return value

None

Description

Service for reporting errors.

8 Appendix B – Access register table

8.1 PERI

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
TIMEOUT_CTL	31:0	Word (32 bits)	0x00000000 timeout value	Timeout control register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FFFF	0x0000**** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
DIV_CMD	31:0	Word (32 bits)	Depends on the configuration value.	Divider command register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CLOCK_CTL	31:0	Word (32 bits)	0x00000000 (PCLK divider type << 8) (PCLK divider index)	Clock control register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x000003FF	0x00000*** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
DIV_8_CTL	31:0	Word (32 bits)	0x00000000 (integer divider value << 8)	Divider control (for 8.0 divider) register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FF01	0x0000**0* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
DIV_16_CTL	31:0	Word (32 bits)	0x00000000 (integer divider value << 8)	Divider control (for 16.0 divider) register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00FFFF01	0x00****0* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
DIV_16_5_CTL	31:0	Word (32 bits)	0x00000000 (integer divider value << 8) (fractional divider value << 3)	Divider control (for 16.5 divider) register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00FFFFF9	0x00***** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
DIV_24_5_CTL	31:0	Word (32 bits)	0x00000000 (integer divider value << 8) (fractional divider value << 3)	Divider control (for 24.5 divider) register	Mcu_Init Mcu_InitClock Mcu_SetMode	0xFFFFFFFF9	0x***** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
PERI_GROUP_STRUCTURE.CLOCK_CTL	31:0	Word (32 bits)	0x00000000 (divider value << 8)	Clock control of the peripheral group register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FF00	0x0000**00 (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
PERI_GROUP_STRUCTURE.SL_CTL	31:0	Word (32 bits)	0x00000000 (slave enable << slave n) (n = 0 - 15)	Peripheral group, slave n disable	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FFFF	0x0000**** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
PCLK_GROUP.DIV_CMD	31:0	Word (32 bits)	Depends on the configuration value.	Divider command register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
PCLK_GROUP.CLOCK_CTL	31:0	Word (32 bits)	0x00000000 (PCLK divider type << 8) (PCLK divider index)	Clock control register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x000003FF	0x00000*** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
PCLK_GROUP.DIV_8_CTL	31:0	Word (32 bits)	0x00000000 (integer divider value << 8)	Divider control (for 8.0 divider) register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FF01	0x0000**0* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
PCLK_GROUP.DIV_16_CTL	31:0	Word (32 bits)	0x00000000 (integer divider value << 8)	Divider control (for 16.0 divider) register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00FFFF01	0x00****0* (After Mcu_Init, Mcu_InitClock and

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
							Mcu_SetMode. Digit * depends on the configuration value.)
PCLK_GROUP.DIV_16_5_CTL	31:0	Word (32 bits)	0x00000000 (integer divider value << 8) (fractional divider value << 3)	Divider control (for 16.5 divider) register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00FFFFFF9	0x00***** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
PCLK_GROUP.DIV_24_5_CTL	31:0	Word (32 bits)	0x00000000 (integer divider value << 8) (fractional divider value << 3)	Divider control (for 24.5 divider) register	Mcu_Init Mcu_InitClock Mcu_SetMode	0xFFFFFFFF9	0x***** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)

8.2 CPUSS

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
IDENTITY	31:0	Word (32 bits)	-	Identity	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CM4_STATUS	31:0	Word (32 bits)	-	CM4 status	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CM4_CLOCK_CTL	31:0	Word (32 bits)	0x00000000 (fast clock divider value << 8)	CM4 clock control	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FF00	0x0000**00 (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CM0_CTL	31:0	Word (32 bits)	-	CM0+ control	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
CM0_STATUS	31:0	Word (32 bits)	-	CM0+ status	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CM0_CLOCK_CTL	31:0	Word (32 bits)	0x00000000 (peri clock divider value << 24) (slow clock divider value << 8)	CM0+ clock control	Mcu_Init Mcu_InitClock Mcu_SetMode	0xFF00FF00	0x**00**00 (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CM4_PWR_CTL	31:0	Word (32 bits)	0x00000000 (register key << 16) power mode	CM4 power control	Mcu_SetMode	0x00000003	0x0000000* (After Mcu_SetMode. Digit * depends on the configuration value.)
CM4_PWR_DELAY_CTL	31:0	Word (32 bits)	0x00000000 power up delay	CM4 power control	Mcu_SetMode	0x000003FF	0x00000*** (After Mcu_SetMode. Digit * depends on the configuration value.)
RAM0_CTL	31:0	Word (32 bits)	0x00000000 (wait cycle for fast domain << 8) (wait cycle for slow domain)	RAM 0 control	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00000303	0x00000*0* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
RAM0_STATUS	31:0	Word (32 bits)	-	RAM 0 status	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
RAM0_PWR_MACRO_CTL	31:0	Word (32 bits)	0x00000000 (register key << 16) power mode	RAM 0 power control	Mcu_SetMode	0x00000003	0x0000000* (After Mcu_SetMode. Digit * depends on the configuration value.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
RAM1_CTL	31:0	Word (32 bits)	0x00000000 (wait cycle for fast domain << 8) (wait cycle for slow domain)	RAM 1 control	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00000303	0x00000*0* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
RAM1_STATUS	31:0	Word (32 bits)	-	RAM 1 status	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
RAM1_PWR_CTL	31:0	Word (32 bits)	0x00000000 (register key << 16) power mode	RAM 1 power control	Mcu_SetMode	0x00000003	0x0000000* (After Mcu_SetMode. Digit * depends on the configuration value.)
RAM2_CTL	31:0	Word (32 bits)	0x00000000 (wait cycle for fast domain << 8) (wait cycle for slow domain)	RAM 2 control	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00000303	0x00000*0* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
RAM2_STATUS	31:0	Word (32 bits)	-	RAM 2 status	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
RAM2_PWR_CTL	31:0	Word (32 bits)	0x00000000 (register key << 16) power mode	RAM 2 power control	Mcu_SetMode	0x00000003	0x0000000* (After Mcu_SetMode. Digit * depends on the configuration value.)
RAM_PWR_DELAY_CTL	31:0	Word (32 bits)	0x00000000 power up delay	Power up delay used for all SRAM power domains	Mcu_SetMode	0x000003FF	0x00000*** (After Mcu_SetMode. Digit * depends on the configuration value.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
ROM_CTL	31:0	Word (32 bits)	0x00000000 (wait cycle for fast domain <= 8) (wait cycle for slow domain)	ROM control	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00000303	0x00000*0* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
SYSTICK_CTL	31:0	Word (32 bits)	-	SysTick timer control	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CM0_SYSTEM_INT_CTL	31:0	Word (32 bits)	-	CM0+ system interrupt control	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CM4_SYSTEM_INT_CTL	31:0	Word (32 bits)	-	CM4 system interrupt control	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CM7_0_STATUS	31:0	Word (32 bits)	-	CM7_0 status	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
FAST_0_CLOCK_CTL	31:0	Word (32 bits)	0x00000000 (fast 0 clock integer divider value <= 8) (fast 0 clock fractional divider value <= 3)	Fast 0 clock control	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FFF8	0x0000**** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
TRC_DBG_CLOCK_CTL	31:0	Word (32 bits)	0x00000000 (trace debug clock divider value <= 8)	Trace debug clock control	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FF00	0x0000**00 (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CM7_1_STATUS	31:0	Word (32 bits)	-	CM7_1 status	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
FAST_1_CLOCK_CTL	31:0	Word (32 bits)	0x00000000 (fast 1 clock integer divider value << 8) (fast 1 clock fractional divider value << 3)	Fast 1 clock control	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FFF8	0x0000**** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
SLOW_CLOCK_CTL	31:0	Word (32 bits)	0x00000000 (slow clock divider value << 8)	Slow clock control	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FF00	0x0000**00 (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
PERI_CLOCK_CTL	31:0	Word (32 bits)	0x00000000 (peri clock divider value << 8)	Peripheral clock control	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FF00	0x0000**00 (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
MEM_CLOCK_CTL	31:0	Word (32 bits)	0x00000000 (mem clock divider value << 8)	Memory clock control	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FF00	0x0000**00 (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CM7_0_PWR_CTL	31:0	Word (32 bits)	0x00000000 (register key << 16) power mode	CM7_0 power control	Mcu_SetMode	0x00000003	0x0000000* (After Mcu_SetMode. Digit * depends on the configuration value.)
CM7_0_PWR_DELAY_CTL	31:0	Word (32 bits)	0x00000000 power up delay	CM7_0 power control	Mcu_SetMode	0x000003FF	0x00000*** (After Mcu_SetMode. Digit * depends on the configuration value.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
CM7_1_PWR_CTL	31:0	Word (32 bits)	0x00000000 (register key << 16) power mode	CM7_1 power control	Mcu_SetMode	0x00000003	0x0000000* (After Mcu_SetMode. Digit * depends on the configuration value.)
CM7_1_PWR_DELAY_CTL	31:0	Word (32 bits)	0x00000000 power-up delay	CM7_1 power control	Mcu_SetMode	0x000003FF	0x00000*** (After Mcu_SetMode. Digit * depends on the configuration value.)
CM7_0_SYSTEM_INT_CTL	31:0	Word (32 bits)	-	CM7_0 system interrupt control	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CM7_1_SYSTEM_INT_CTL	31:0	Word (32 bits)	-	CM7_1 system interrupt control	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)

8.3 DW

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
CTL	31:0	Word (32 bits)	0x00000000 (DW enable << 31)	Control	Mcu_SetMode	0x80000000	0x*0000000 (After Mcu_SetMode. Digit * depends on the configuration value.)

8.4 DMAC

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
CTL	31:0	Word (32 bits)	0x00000000 (DMAC enable << 31)	Control	Mcu_SetMode	0x80000000	0x*0000000 (After Mcu_SetMode. Digit * depends on the configuration value.)

8.5 FLASHC

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
FLASH_CTL	31:0	Word (32 bits)	0x00000000 wait cycle	Flash control	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000000F	0x0000000* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)

8.6 SRSS

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
PWR_LVD_CTL	31:0	Word (32 bits)	Depends on the configuration value.	High voltage / low voltage detector (HVLVD) configuration register	Mcu_SetMode	0x0007DF00	0x000***00 (After Mcu_SetMode. Digit * depends on the configuration value.)
PWR_LVD_CTL2	31:0	Word (32 bits)	Depends on the configuration value.	High voltage / low voltage detector (HVLVD) configuration register #2	Mcu_SetMode	0x0007DF00	0x000***00 (After Mcu_SetMode. Digit * depends on the configuration value.)
CLK_DSI_SELECT	31:0	Word (32 bits)	0x00000000 DSI source	Clock DSI select register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000001F	0x000000** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CLK_OUTPUT_FAST	31:0	Word (32 bits)	Depends on the configuration value.	Fast clock output select register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0FFF0FFF	0x0***0*** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
CLK_OUTPUT_SLOW	31:0	Word (32 bits)	Depends on the configuration value.	Slow clock output select register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x000000FF	0x000000** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CLK_CAL_CNT1	31:0	Word (32 bits)	-	Clock calibration counter 1	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CLK_CAL_CNT2	31:0	Word (32 bits)	-	Clock calibration counter 2	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
SRSS_INTR	31:0	Word (32 bits)	0x00000000 (HVLVD2 interrupt << 2) (HVLVD1 interrupt << 1)	SRSS interrupt register	Mcu_SetMode Mcu_Lvd_Isr_Cat1 Mcu_Lvd_Isr_Cat2	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
SRSS_INTR_SET	31:0	Word (32 bits)	-	SRSS interrupt set register	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
SRSS_INTR_MASK	31:0	Word (32 bits)	0x00000000 (HVLVD2 interrupt << 2) (HVLVD1 interrupt << 1)	SRSS interrupt mask register	Mcu_SetMode	0x00000003	0x0000000* (After Mcu_SetMode. Digit * depends on the configuration value.)
SRSS_INTR_MASKE D	31:0	Word (32 bits)	-	SRSS interrupt masked register	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
PWR_CTL	31:0	Word (32 bits)	-	Power mode control	Read-only.	0x00000000 (monitoring is not required.)	0x00000000 (Monitoring is not required.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
PWR_CTL2	31:0	Word (32 bits)	Depends on the configuration value.	Power mode control 2	Mcu_SetMode	0x81100011	0x***000** (After Mcu_SetMode. Digit * depends on the configuration value.)
PWR_HIBERNATE	31:0	Word (32 bits)	Depends on the configuration value.	HIBERNATE mode register	Mcu_SetMode	0xBFFEFFFF	0x****3AFF (After Mcu_SetMode to Hibernate mode. Digit * depends the on the configuration value.) 0x00000000 (After Mcu_SetMode to Sleep or DeepSleep mode.)
PWR_HIB_WAKE_CTL	31:0	Word (32 bits)	Depends on the configuration value.	HIBERNATE Wakeup mask register	Mcu_SetMode	0xE0FFFFFF	0x***** (After Mcu_SetMode. Digit * depends on the configuration value.)
PWR_HIB_WAKE_CTL2	31:0	Word (32 bits)	Depends on the configuration value.	HIBERNATE Wakeup polarity register	Mcu_SetMode	0x00FFFFFF	0x***** (After Mcu_SetMode. Digit * depends on the configuration value.)
PWR_HIB_WAKE_CAUSE	31:0	Word (32 bits)	0xE0FFFFFF	HIBERNATE Wakeup cause register	Mcu_SetMode	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
PWR_SSV_CTL	31:0	Word (32 bits)	Depends on the configuration value.	Supply supervision control register	Mcu_SetMode	0x09D909D9	0x***** (After Mcu_SetMode. Digit * depends on the configuration value.)
PWR_REGHC_STATUSES	31:0	Word (32 bits)	-	REGHC status register	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
PWR_REGHC_CTL	31:0	Word (32 bits)	-	REGHC control register	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
PWR_REGHC_CTL2	31:0	Word (32 bits)	Depends on the configuration value.	REGHC control register 2	Mcu_SetMode	0x80000000	0x*00000000 (After Mcu_SetMode. Digit * depends on the configuration value.)
PWR_REGHC_CTL4	31:0	Word (32 bits)	Depends on the configuration value.	REGHC control register 4	Mcu_SetMode	0xC0000000	0x*00000000 (After Mcu_SetMode. Digit * depends on the configuration value.)
PWR_PMIC_STATUS	31:0	Word (32 bits)	-	PMIC status register	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
PWR_PMIC_CTL	31:0	Word (32 bits)	-	PMIC control register	Read-only	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
PWR_PMIC_CTL2	31:0	Word (32 bits)	Depends on the configuration value.	PMIC control register 2	Mcu_SetMode	0x80000000	0x*00000000 (After Mcu_SetMode. Digit * depends on the configuration value.)
PWR_PMIC_CTL4	31:0	Word (32 bits)	Depends on the configuration value.	PMIC control register 4	Mcu_SetMode	0xC0000000	0x*00000000 (After Mcu_SetMode. Digit * depends on the configuration value.)
TST_DDFT_FAST_CTL	31:0	Word (32 bits)	Depends on the configuration value.	Fast digital DFT control register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x66003F3F	0x**00**** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
TST_DDFT_SLOW_CTL	31:0	Word (32 bits)	Depends on the configuration value.	Slow digital DFT control register	Mcu_Init Mcu_InitClock Mcu_SetMode	0xC0009F9F	0x*000**** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CLK_PATH_SELECT	31:0	Word (32 bits)	0x00000000 clock path source	Clock path select register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00000007	0x0000000* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CLK_ROOT_SELECT	31:0	Word (32 bits)	0x00000000 (root clock enable << 31) (root clock direct mux << 8) (root clock divider value << 4) root clock source	Clock root select register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x8000013F	0x*0000**** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CSV_HF_STRUCTS. CSV_ACT_STRUCT. REF_CTL	31:0	Word (32 bits)	0x00000000 (CSV enable << 31) (CSV action << 30) CSV startup delay	Clock supervision reference control for root clocks	Mcu_Init Mcu_InitClock Mcu_SetMode	0xC000FFFF	0x*000**** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CSV_HF_STRUCTS. CSV_ACT_STRUCT. REF_LIMIT	31:0	Word (32 bits)	0x00000000 (CSV upper threshold << 16) CSV lower threshold	Clock supervision reference limits for root clocks	Mcu_Init Mcu_InitClock Mcu_SetMode	0xFFFFFFFF	0x****** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)

Appendix B – Access register table

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
CSV_HF_STRUCTS. CSV_ACT_STRUCT. MON_CTL	31:0	Word (32 bits)	0x00000000 CSV period	Clock supervision monitor control for root clocks	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FFFF	0x0000**** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CSV_HF_STRUCTS. CSV_ACT_STRUCT. CNT_STAT	31:0	Word (32 bits)	-	Clock supervision counters for root clocks	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CLK_SELECT	31:0	Word (32 bits)	Depends on the configuration value.	Clock selection register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FF03	0x0000**0* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CLK_TIMER_CTL	31:0	Word (32 bits)	Depends on the configuration value.	Timer clock control register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x80FF0301	0x*0**0*0* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CLK_ILO0_CONFIG	31:0	Word (32 bits)	0x00000000 (ILO0 enable << 31) (ILO0 monitor enable << 30) ILO0 backup enable	ILO0 configuration	Mcu_Init Mcu_InitClock Mcu_SetMode	0xC0000001	0x*000000* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CLK_ILO1_CONFIG	31:0	Word (32 bits)	0x00000000 (ILO1 enable << 31) (ILO1 monitor enable << 30)	ILO1 configuration	Mcu_Init Mcu_InitClock Mcu_SetMode	0xC0000000	0x*0000000 (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
CLK_IMO_CONFIG	31:0	Word (32 bits)	0x00000000 (IMO enable << 31)	IMO configuration	Mcu_Init Mcu_InitClock Mcu_SetMode	0x80000000	0x*0000000 (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CLK_ECO_CONFIG	31:0	Word (32 bits)	0x00000000 (ECO enable << 31) (ECO divider enable << 28) (ECO divider disable << 27) (AGC enable << 1)	ECO configuration register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x98000002	0x**00000* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CLK_ECO_PRESCALE	31:0	Word (32 bits)	0x00000000 (ECO integer divider value << 16) (ECO fractional divider value << 8)	ECO prescaler configuration register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x03FFFF00	0x0*****00 (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CLK_ECO_STATUS	31:0	Word (32 bits)	-	ECO status register	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CLK_FLL_CONFIG	31:0	Word (32 bits)	Depends on the configuration value.	FLL configuration register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x8103FFFF	0x**0***** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CLK_FLL_CONFIG2	31:0	Word (32 bits)	Depends on the configuration value.	FLL configuration register 2	Mcu_Init Mcu_InitClock Mcu_SetMode	0xFFFF1FFF	0x***** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
							depends on the configuration value.)
CLK_FLL_CONFIG3	31:0	Word (32 bits)	Depends on the configuration value.	FLL configuration register 3	Mcu_Init Mcu_InitClock Mcu_SetMode Mcu_Distribut ePl1Clock	0x301FFFFF	0x*0***** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.) 0x*0000000 (After Mcu_DistributePl1Clock. Digit * depends on the configuration value.)
CLK_FLL_CONFIG4	31:0	Word (32 bits)	Depends on the configuration value.	FLL configuration register 4	Mcu_Init Mcu_InitClock Mcu_SetMode	0xC1FF07FF	0x****0*** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CLK_FLL_STATUS	31:0	Word (32 bits)	-	FLL status register	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CLK_ECO_CONFIG2	31:0	Word (32 bits)	Depends on the configuration value.	ECO configuration register 2	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00007FF7	0x0000**** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CLK_PLL_CONFIG	31:0	Word (32 bits)	Depends on the configuration value.	PLL configuration register	Mcu_Init Mcu_InitClock Mcu_SetMode Mcu_Distribut ePl1Clock	0xB81F1F7F	0x***** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit *

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
							depends on the configuration value.) 0x*00000000 (After Mcu_DistributePl lClock. Digit * depends on the configuration value.)
CLK_PLL_STATUS	31:0	Word (32 bits)	-	PLL status register	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CSV_REF_SEL	31:0	Word (32 bits)	0x00000000 CSV reference clock	Select CSV reference clock for Active domain	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00000007	0x0000000* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CSV_REF_STRUCT. CSV_ACT_STRUCT. REF_CTL	31:0	Word (32 bits)	0x00000000 (CSV enable << 31) (CSV action << 30) CSV startup delay	Clock supervision reference control for reference clock	Mcu_Init Mcu_InitClock Mcu_SetMode	0xC00FFFFF	0x*000**** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CSV_REF_STRUCT. CSV_ACT_STRUCT. REF_LIMIT	31:0	Word (32 bits)	0x00000000 (CSV upper threshold << 16) CSV lower threshold	Clock supervision reference limits for reference clock	Mcu_Init Mcu_InitClock Mcu_SetMode	0xFFFFFFFF	0x***** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CSV_REF_STRUCT. CSV_ACT_STRUCT. MON_CTL	31:0	Word (32 bits)	0x00000000 CSV period	Clock supervision monitor control for reference clock	Mcu_Init Mcu_InitClock Mcu_SetMode	0x0000FFFF	0x0000**** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit *

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
							depends on the configuration value.)
CSV_REF_STRUCT. CSV_ACT_STRUCT. CNT_STAT	31:0	Word (32 bits)	-	Clock supervision counters for reference clock	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CSV_LF_STRUCT.C SV_DPSLP_STRUCT. .REF_CTL	31:0	Word (32 bits)	0x00000000 (CSV enable << 31) CSV startup delay	Clock supervision reference control for LF clock	Mcu_Init Mcu_InitClock Mcu_SetMode	0x800001FF	0x*0000*** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CSV_LF_STRUCT.C SV_DPSLP_STRUCT. .REF_LIMIT	31:0	Word (32 bits)	0x00000000 (CSV upper threshold << 16) CSV lower threshold	Clock supervision reference limits for LF clock	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00FF00FF	0x00**00** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CSV_LF_STRUCT.C SV_DPSLP_STRUCT. .MON_CTL	31:0	Word (32 bits)	0x00000000 CSV period	Clock supervision monitor control for LF clock	Mcu_Init Mcu_InitClock Mcu_SetMode	0x000000FF	0x000000** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CSV_LF_STRUCT.C SV_DPSLP_STRUCT. .CNT_STAT	31:0	Word (32 bits)	-	Clock supervision counters for LF clock	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CSV_ILO_STRUCT. CSV_DPSLP_STRUC T.REF_CTL	31:0	Word (32 bits)	0x00000000 (CSV enable << 31) CSV startup delay	Clock supervision reference control for HVILO clock	Mcu_Init Mcu_InitClock Mcu_SetMode	0x800001FF	0x*0000*** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
CSV_ILO_STRUCT. CSV_DPSLP_STRUC T.REF_LIMIT	31:0	Word (32 bits)	0x00000000 (CSV upper threshold <= 16) CSV lower threshold	Clock supervision reference limits for HVILO clock	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00FF00FF	0x00**00** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CSV_ILO_STRUCT. CSV_DPSLP_STRUC T.MON_CTL	31:0	Word (32 bits)	0x00000000 CSV period	Clock supervision monitor control for HVILO clock	Mcu_Init Mcu_InitClock Mcu_SetMode	0x000000FF	0x000000** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CSV_ILO_STRUCT. CSV_DPSLP_STRUC T.CNT_STAT	31:0	Word (32 bits)	-	Clock supervision counters for HVILO clock	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
RES_CAUSE	31:0	Word (32 bits)	0x61FF01FF	Reset cause observation register	Mcu_Init	0x61FF01FF	0x****0*** (After Mcu_Init. Digit * depends on then configuration value.)
RES_CAUSE2	31:0	Word (32 bits)	0x0001FFFF	Reset cause observation register 2	Mcu_Init	0x0001FFFF	0x000***** (After Mcu_Init. Digit * depends on the configuration value.)
WDT_B_STRUCT.LO CK	31:0	Word (32 bits)	0x00000000 lock value	WDT lock register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
PLL400M_STRUCT. CONFIG	31:0	Word (32 bits)	Depends on the configuration value.	400MHz PLL configuration register	Mcu_Init Mcu_InitClock Mcu_SetMode	0xB61F1FFF	0x***** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
PLL400M_STRUCT. CONFIG2	31:0	Word (32 bits)	Depends on the configuration value.	400MHz PLL configuration register 2	Mcu_Init Mcu_InitClock Mcu_SetMode	0xF0FFFFFF	0x*0*****
PLL400M_STRUCT. CONFIG3	31:0	Word (32 bits)	Depends on the configuration value.	400MHz PLL configuration register 3	Mcu_Init Mcu_InitClock Mcu_SetMode	0x910703FF	0x**0*0*** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
PLL400M_STRUCT. STATUS	31:0	Word (32 bits)	-	400MHz PLL status register	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)

8.7 BACKUP

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
CTL	31:0	Word (32 bits)	Depends on the configuration value.	Control	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00013308	0x000***0* (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
STATUS	31:0	Word (32 bits)	-	Status	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
RTC_RW	31:0	Word (32 bits)	0x00000000 0x00000001 0x00000002	RTC read write register	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
LPECO_CTL	31:0	Word (32 bits)	0x00000000 (LPECO enable << 31) (LPECO amplitude detector enable << 30) (LPECO divider enable << 28) (LPECO maximum amplitude << 12) (LPECO frequency range << 8) (LPECO capacitance range << 4)	Low-power external crystal oscillator control	Mcu_Init Mcu_InitClock Mcu_SetMode	0xD0001130	0x*000***0 (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
LPECO_PRESC ALE	31:0	Word (32 bits)	0x00000000 (LPECO integer divider value << 16) (LPECO fractional divider value << 8)	Low-power external crystal oscillator prescaler	Mcu_Init Mcu_InitClock Mcu_SetMode	0x03FFFF00	0x0*****00 (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
LPECO_STATU S	31:0	Word (32 bits)	-	Low-power external crystal oscillator status	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CSV_BAK_STR UCT.CSV_DPS LP_STRUCT.R EF_CTL	31:0	Word (32 bits)	0x00000000 (CSV enable << 31) CSV startup delay	Clock supervision reference control for backup clock	Mcu_Init Mcu_InitClock Mcu_SetMode	0x800001FF	0x*0000*** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CSV_BAK_STR UCT.CSV_DPS LP_STRUCT.R EF_LIMIT	31:0	Word (32 bits)	0x00000000 (CSV upper threshold << 16) CSV lower threshold	Clock supervision reference limits for backup clock	Mcu_Init Mcu_InitClock Mcu_SetMode	0x00FF00FF	0x00**00** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
CSV_BAK_STR UCT.CSV_DPS LP_STRUCT.M ON_CTL	31:0	Word (32 bits)	0x00000000 CSV period	Clock supervision monitor control for backup clock	Mcu_Init Mcu_InitClock Mcu_SetMode	0x000000FF	0x000000** (After Mcu_Init, Mcu_InitClock and Mcu_SetMode. Digit * depends on the configuration value.)
CSV_BAK_STR UCT.CSV_DPS LP_STRUCT.C NT_STAT	31:0	Word (32 bits)	-	Clock supervision counters for backup clock	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)

8.8 CM0P_SCS

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
SYST_CSR	31:0	Word (32 bits)	-	Cortex®-M0+ SysTick control & status	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CPUID	31:0	Word (32 bits)	-	Cortex®-M0+ CPUID register	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
AIRCR	31:0	Word (32 bits)	-	Cortex®-M0+ application interrupt and reset control register	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
SCR	31:0	Word (32 bits)	0x00000000 (pending interrupt enable << 4) (deepsleep enable << 2) (sleep on exit enable << 1)	Cortex®-M0+ system control register	Mcu_SetMode	0x00000016	0x000000** (After Mcu_SetMode. Digit * depends on the configuration value.)

8.9 CM4_SCS

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
ACTLR	31:0	Word (32 bits)	-	Cortex®-M4 Auxiliary control register	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
SYST_CSR	31:0	Word (32 bits)	-	Cortex®-M4 SysTick control and status register	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CPUID	31:0	Word (32 bits)	-	Cortex®-M4 CPUID base register	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
AIRCR	31:0	Word (32 bits)	0x00000000 (register key << 16) (reset request << 2)	Cortex®-M4 application interrupt and reset control register	Mcu_PerformReset	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
SCR	31:0	Word (32 bits)	0x00000000 (pending interrupt enable << 4) (deepsleep enable << 2) (sleep on exit enable << 1)	Cortex®-M4 system control register	Mcu_SetMode	0x00000016	0x000000** (After Mcu_SetMode. Digit * depends on the configuration value.)

8.10 CM7_SCS

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
ACTLR	31:0	Word (32 bits)	-	Cortex®-M7 Auxiliary control register	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
SYST_CSR	31:0	Word (32 bits)	-	Cortex®-M7 SysTick control and status register	Not used.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CPUID	31:0	Word (32 bits)	-	Cortex®-M7 CPUID base register	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)

Register	Bit No.	Access size	Value	Description	Timing	Mask value	Monitoring value
AIRCR	31:0	Word (32 bits)	0x00000000 (register key << 16) (reset request << 2)	Cortex®-M7 application interrupt and reset control register	Mcu_PerformReset	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
SCR	31:0	Word (32 bits)	0x00000000 (pending interrupt enable << 4) (deepsleep enable << 2) (sleep on exit enable << 1)	Cortex®-M7 system control register	Mcu_SetMode	0x00000016	0x000000** (After Mcu_SetMode. Digit * depends on the configuration value.)
CCR	31:0	Word (32 bits)	0x00000000 (I-cache enable << 17) (D-cache enable << 16)	Cortex®-M7 configuration and control register	Mcu_SetMode Mcu_PerformReset	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CCSIDR	31:0	Word (32 bits)	-	Cortex®-M7 cache size ID register	Read-only.	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
CSSELR	31:0	Word (32 bits)	0x00000000	Cortex®-M7 cache size selection register	Mcu_SetMode Mcu_PerformReset	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
ICIALLU	31:0	Word (32 bits)	0x00000000	Cortex®-M7 instruction cache invalidate all	Mcu_SetMode Mcu_PerformReset	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
DCISW	31:0	Word (32 bits)	0x00000000 (way << 30) (set << 5)	Cortex®-M7 data cache invalidate by set/way	Mcu_SetMode Mcu_PerformReset	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)
DCCSW	31:0	Word (32 bits)	0x00000000 (way << 30) (set << 5)	Cortex®-M7 data cache clean by set/way	Mcu_SetMode Mcu_PerformReset	0x00000000 (Monitoring is not required.)	0x00000000 (Monitoring is not required.)

Revision history

Revision	Issue date	Description of change
**	2018-05-24	New spec.
*A	2018-12-21	Added acronyms CM7, DMA, LPECO, OCD, PFD, PMIC, REGHC, and SSCG to Table 1 in Glossary. Added the documents 002-24401 and 002-24402 in Hardware Documentation. Deleted the datasheet 002-18043 from Hardware Documentation. Added reset reasons to Table 4-1 in 4.8 Mcu Published Information Added following sections. 4.5.1.3 Mcu ECO Trim Settings 4.5.1.4 Mcu LPECO Settings 4.5.1.5 Mcu LPECO Prescaler Settings 4.5.2.4 Mcu SSCG PLL Clock Settings 4.5.2.6 Mcu PCLK Group Settings 4.6.4 Mcu REGHC Settings 4.6.5 Mcu DMA Settings B.1.3 DW B.1.4 DMAC B.1.10 CM7_SCS Added description about following configuration parameters. McuEnableCacheFlushBeforeReset in 4.2 Mcu Module Configuration McuForcedResetEnable in 4.2 Mcu Module Configuration McuRam2RetainBeforeReset in 4.2 Mcu Module Configuration McuWcoStopForUpdate in 4.5.1.8 Mcu WCO Clock Settings McuFast1ClockFrequency in 4.5.2 Mcu Clock Settings McuFast1ClockDivision in 4.5.2 Mcu Clock Settings McuMemClockFrequency in 4.5.2 Mcu Clock Settings McuMemClockDivision in 4.5.2 Mcu Clock Settings McuTrcDbgClockFrequency in 4.5.2 Mcu Clock Settings McuTrcDbgClockDivision in 4.5.2 Mcu Clock Settings McuFastRam2WaitCycle in 4.5.2 Mcu Clock Settings McuSlowRam2WaitCycle in 4.5.2 Mcu Clock Settings McuClockRootSscgPllRef in 4.5.2.5 Mcu Clock Root Settings McuPumpClockSscgPllRef in 4.5.2.12 Mcu Pump Clock Settings McuMainCore1PowerMode in 4.6 Mcu Mode Settings Configuration McuEnableCacheFlushBeforeModeChange in 4.6 Mcu Mode Settings Configuration McuMainCore1PowerUpDelay in 4.6 Mcu Mode Settings Configuration McuRam2PowerMode in 4.6 Mcu Mode Settings Configuration Added note for following configuration parameters. McuFllAutoDistributeEnable in 4.5.2.2 Mcu FLL Clock Settings McuLfClockSource in 4.5.2.11 Mcu LF Clock Settings McuClockOutput0Enable in 4.5.2.14 Mcu Clock Output Settings

Revision history

Revision	Issue date	Description of change
		McuClockOutput1Enable in 4.5.2.14 Mcu Clock Output Settings Added value for following configuration parameters. McuBackupClockSource in 4.5.2 Mcu Clock Settings McuClockPathSource in 4.5.2.1 Mcu Clock Path Settings McuFllSource in 4.5.2.2 Mcu FLL Clock Settings McuPllSource in 4.5.2.3 Mcu PLL Clock Settings McuClockRootSource in 4.5.2.5 Mcu Clock Root Settings McuTargetCpu in 4.6 Mcu Mode Settings Configuration Modified value for following configuration parameters. McuFast0ClockDivision to 1.0 - 256.96875 in 4.5.2 Mcu Clock Settings Modified name of following configuration parameters. McuFastClockFrequency to McuFast0ClockFrequency in 4.5.2 Mcu Clock Settings McuFastClockDivision to McuFast0ClockDivision in 4.5.2 Mcu Clock Settings McuCm4PowerMode to McuMainCore0PowerMode in 4.6 Mcu Mode Settings Configuration McuCm4PoewrUpDelay to McuMainCore0PowerUpDelay in 4.6 Mcu Mode Settings Configuration Added notes about the build for low-power mode in 2.4 Starting the Build Process Added notes about the REGHC and the DMA settings in 5.3 Mcu Mode Added notes about the acceptable APIs on the slave CPU core in 5.3 Mcu Mode Added description of some registers in following sections. B.1.1 PERI B.1.2 CPUSS B.1.6 SRSS B.1.7 BACKUP Modified some clerical errors.
*B	2019-06-14	Added acronyms VADJ to Table 1 in Glossary. Added description about McuPmicSettings in 4.6 Mcu Mode Settings Configuration. Added following section. 4.6.5 Mcu PMIC Settings Added description about following configuration parameters. McuRegHcOnDeepSleepEnable in 4.6.4 Mcu REGHC Settings McuRegHcPmicVadjDisable in 4.6.4 Mcu REGHC Settings Modified note for following configuration parameter. McuRegHcSettings in 4.6.4 Mcu REGHC Settings Added and modified description of some registers in following sections. B.1.6 SRSS B.1.7 BACKUP

Revision history

Revision	Issue date	Description of change
		Updated hardware documentation information.
*C	2019-10-29	Added note for following configuration parameters. McuRam0Macro<n>PowerMode in 4.6 Mcu Mode Settings Configuration McuRam1PowerMode in 4.6 Mcu Mode Settings Configuration McuRam2PowerMode in 4.6 Mcu Mode Settings Configuration
*D	2020-07-27	Added note for Mcullo0MonitorEnable in 4.5.1.6 Mcu ILO Clock Settings Added note for Mcullo1MonitorEnable in 4.5.1.7 Mcu ILO1 Clock Settings Added value and notes for McuSscgPllModulationRate in 4.5.2.4 Mcu SSCG PLL Clock Settings. Modified notes for McuMainCore1PowerMode in 4.6 Mcu Mode Settings Configuration.
*E	2020-09-05	Changed a memmap file include folder in section "Memory Mapping". Modified the value of McuEcoAmplitudeTrimValue and McuEcoWatchdogTrimValue in 4.5.1.3 MCU ECO Trim Settings.. Added note for McuDeepSleepRegulatorDisable in 4.6 Mcu Mode Settings Configuration.
*F	2020-11-20	Added section 5.8 APIs Require Privileged Execution. MOVED TO INFINEON TEMPLATE.
*G	2021-02-15	Added the error case description in section 5.4. Added note for McuCsvAction in 4.5.2.15 MCU Clock Supervisor Settings Added note for McuHvLvdAction in 4.6.2 MCU HVLVD Settings Added note for McuVddaBodAction and McuVddaOvdAction in 4.6.3 MCU Supply Supervision Settings Added the section 6.3 Fault report structure. Add description about following configuration parameters. McuHibernateWakeupByBackupCsvEnable in 4.6.1 MCU HIBRENATE Mode Settings McuHibernateWakeupSenseMode in 4.6.1 MCU HIBRENATE Mode Settings Modified following configuration parameters. McuHibernateWakeupByWakeupPin<n>Enable in 4.6.1 MCU HIBRENATE Mode Settings McuHibernateWakeupPin<n>Polarity in 4.6.1 MCU HIBRENATE Mode Settings Added value for following configuration parameters. McuCsvClock in 4.5.2.15 MCU Clock Supervisor Settings McuCsvStartupDelay in 4.5.2.15 MCU Clock Supervisor Settings Added note for following configuration parameters. McuSscgPllModulationDitheringEnable in 4.5.2.4 MCU SSCG PLL Clock Settings McuPumpClockSettings in 4.5.2.12 MCU Pump Clock Settings

Revision history

Revision	Issue date	Description of change
		McuCsvStartupDelay in 4.5.2.15 MCU Clock Supervisor Settings Added description for following registers. PWR_HIB_WAKE_CTL in 8.6 SRSS PWR_HIB_WAKE_CTL2 in 8.6 SRSS PWR_HIB_WAKE_CAUSE in 8.6 SRSS CSV_BAK_STRUCT.CSV_DPSLP_STRUCT.REF_CTL in 8.7 BACKUP CSV_BAK_STRUCT.CSV_DPSLP_STRUCT.REF_LIMIT in 8.7 BACKUP CSV_BAK_STRUCT.CSV_DPSLP_STRUCT.MON_CTL in 8.7 BACKUP CSV_BAK_STRUCT.CSV_DPSLP_STRUCT.CNT_STAT in 8.7 BACKUP Modified description for following registers PWR_HIBERNATE in 8.6 SRSS CSV_LF_STRUCT.CSV_DPSLP_STRUCT.REF_CTL in 8.6 SRSS CSV_ILO_STRUCT.CSV_DPSLP_STRUCT.REF_CTL in 8.6 SRSS
*H	2021-05-19	Modified the notes for following configuration parameters: McuDefaultClockSetting in 4.2 MCU Module Configuration McuFast0ClockFrequency, McuFast1ClockFrequency, and McuSlowClockFrequency in 4.5.2 MCU Clock Settings McuSscgPllFrequency in 4.5.2.4 MCU SSCG PLL Clock Settings McuPclkFrequency in 4.5.2.8 MCU PCLK Settings McuPeriGroupClockFrequency in 4.5.2.9 MCU Peripheral Group Settings McuPeriGroupSlaveEnable in 4.5.2.10 MCU Peripheral Group Slave Settings McuLfClockSource in 4.5.2.11 MCU LF Clock Settings McuFreezeLoRelease in 4.6 MCU Mode Settings Configuration Deleted note for following configuration parameters. MCU_E_CLOCK_FAILURE and MCU_E_RESET_FAILURE in 4.4 MCU DEM Event Parameter References McuFllAutoDistributeEnable and McuFllAutoDistributeType in 4.5.2.2 MCU FLL Clock Settings McuHvLvdType in 4.6.2 MCU HVLVD Settings
*I	2021-05-25	Added a note for following configuration parameters. McuRegHcSettings in 4.6.4 MCU REGHC Settings McuPmicSettings in 4.6.5 MCU PMIC Settings
*J	2021-06-25	Added the definition of WFI in Abbreviations and definitions Added note about Hibernate mode entry in section 5.3 MCU Mode Deleted value for McuSscgPllModulationMode in 4.5.2.4 MCU SSCG PLL Clock Settings Added note for following configuration parameters. McuImoEnable in 4.5.1 MCU Clock Input McuFllAutoDistributeEnable and McuFllAutoDistributeType in 4.5.2.2 MCU FLL Clock Settings McuLinearCoreRegulatorDisable in 4.6 MCU Mode Settings Configuration

Revision history

Revision	Issue date	Description of change
		Deleted note for McuVoltageReferenceBufferDisable in 4.6 MCU Mode Settings Configuration
*K	2021-08-19	Added a note in 6.2 Interrupts

Trademarks

All referenced product or service names and trademarks are the property of their respective owners.

Edition 2021-08-19

Published by

**Infineon Technologies AG
81726 Munich, Germany**

**© 2021 Infineon Technologies AG.
All Rights Reserved.**

Do you have a question about this document?

Go to www.cypress.com/support

Document reference

002-23349 Rev. *K

IMPORTANT NOTICE

The information given in this document shall in no event be regarded as a guarantee of conditions or characteristics ("Beschaffheitsgarantie").

With respect to any examples, hints or any typical values stated herein and/or any information regarding the application of the product, Infineon Technologies hereby disclaims any and all warranties and liabilities of any kind, including without limitation warranties of non-infringement of intellectual property rights of any third party.

In addition, any information given in this document is subject to customer's compliance with its obligations stated in this document and any applicable legal requirements, norms and standards concerning customer's products and any use of the product of Infineon Technologies in customer's applications.

The data contained in this document is exclusively intended for technically trained staff. It is the responsibility of customer's technical departments to evaluate the suitability of the product for the intended application and the completeness of the product information given in this document with respect to such application.

For further information on the product, technology, delivery terms and conditions and prices please contact your nearest Infineon Technologies office (www.infineon.com).

WARNINGS

Due to technical requirements products may contain dangerous substances. For information on the types in question please contact your nearest Infineon Technologies office.

Except as otherwise explicitly approved by Infineon Technologies in a written document signed by authorized representatives of Infineon Technologies, Infineon Technologies' products may not be used in any applications where a failure of the product or any consequences of the use thereof can reasonably be expected to result in personal injury.